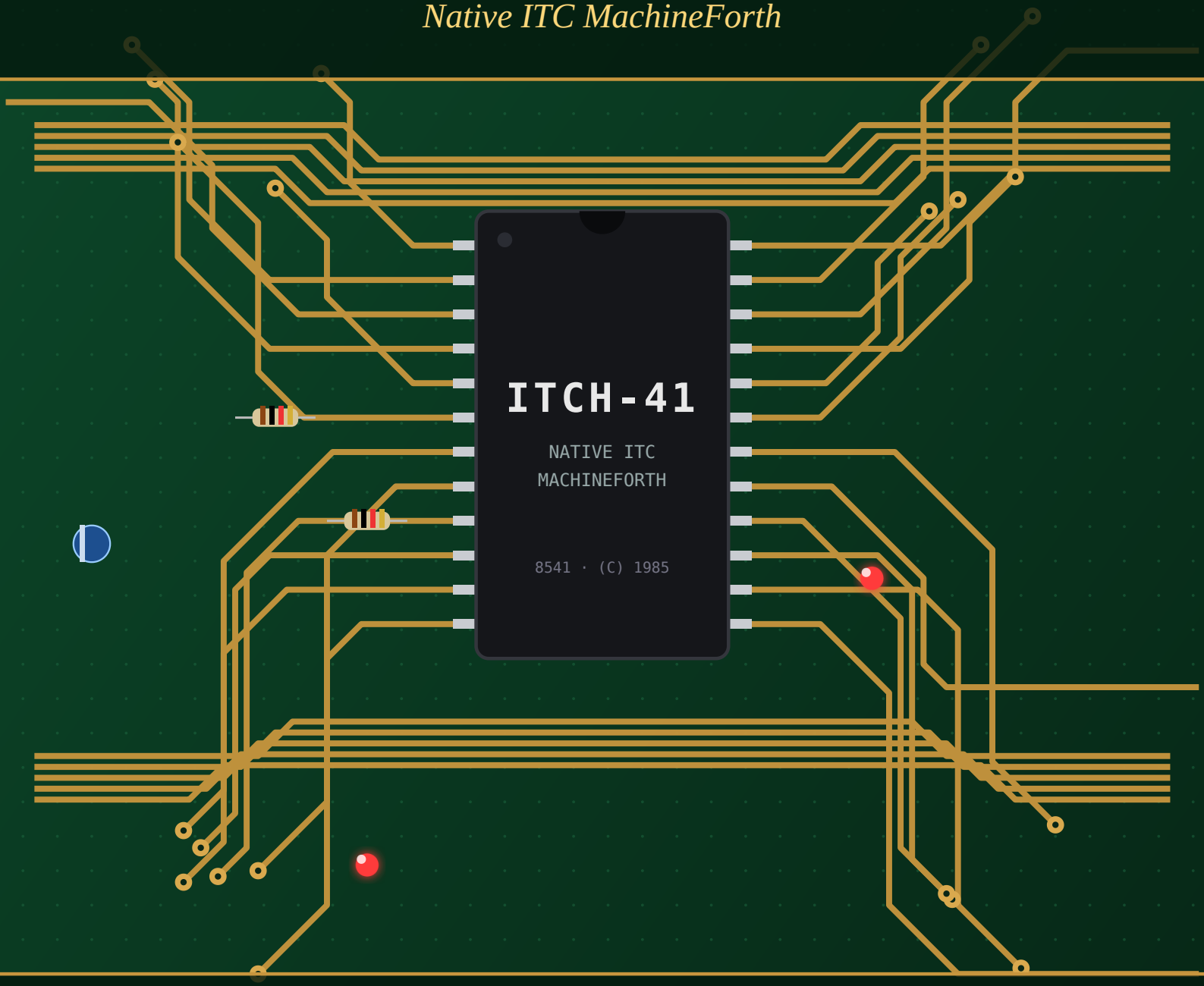
FORTH · 16-BIT · INDIRECT THREADED

ITCH PRIMITIVES

A BARE-METAL REFERENCE

Native ITC MachineForth



41 PRIMITIVES · 65C02 · x86-64 · ARM64 · RISC-V

every implementation proven byte for byte against the machine

by Jimmy Söderman

ITCH Primitives: A Bare-Metal Reference

The 41 primitives of the ITCH kernel (ITCH-TINY.html), explained one by one and implemented for the 65C02, x86-64, ARM64 and RISC-V. Every implementation here was run against the JavaScript machine and left RAM byte-identical to it; every number was measured.

Contents

1. The short answer
2. The machine every port shares
3. How the 2x2 design shaped this primitive set
4. The inner interpreter
5. The stack macros
6. The 41 primitives
7. Shared routines
8. Implementation proof
9. Trade-offs, stated plainly

1. The short answer

Is each primitive 1–3 opcodes? For the very simplest, on a 64-bit CPU, nearly: BRANCH is 2 x86-64 instructions and WAIT is 2. But ITCH checks every stack move, so the everyday primitives that pop and push (+ DUP SWAP @ ! >R EXIT …) are 6–31 instructions on the 64-bit CPUs, and 14–49 on the 8-bit 6502 (43–150 cycles), where every 16-bit cell takes two loads and two stores. The 2x2 compiler primitives are 9–81, except ; and (CLOSE), which walk a wordlist; the seed's text words walk strings and whole wordlists and run to hundreds or thousands per call. All of it is small: **the whole kernel, 41 primitives plus the inner interpreter and the seed, is 4.2–5.1 KB of machine code on every CPU.**

	65C02	x86-64	ARM64	RISC-V
kernel size (bytes)	4,451	4,178	4,452	5,136
instructions per Forth step, on average	65	39	46	50
… of which the primitive's own body	26	12	14	14
… of which NEXT and dispatch	39	27	32	36
cycles per Forth step (6502)	196			

Measured over the whole proof workload: 1,933,150 Forth steps, identical on all four CPUs.

So the primitives themselves are small and cheap. **The cost that matters is the dispatch around them**: about 27 instructions per step on x86-64 and 39 on the 6502. That is the price of four deliberate ITCH choices: the machine's registers live in RAM, every step checks the STOP register (resumable errors) and the return stack (QUIT returning), a word's xt is its header (so EXECUTE finds the code through the name's length), and the kernel can be stopped after any step and resumed. Section 9 says what each costs and what a speed-first port would change. At 1 MHz the faithful 6502 kernel runs about 5,100 Forth steps a second.

2. The machine every port shares

Every port runs the **same 64K RAM image**, booted by the JavaScript kernel, with the same memory map. What differs is only which CPU executes the kernel's routines.

Addresses	What	Notes
$00–$3F	the register page	CSP $02, >IN $04, #TIB $06, LATEST $08, HERE $0A, SP $0C, RP $0E, IP $12, SRC $18, MSG $1A, STOP $22, SELF $2A, CXP $2C, CFP $2E, the wordlists $30 $32 $34, CURRENT $36

Addresses	What	Notes
$04C0	data stack, 256 bytes	depth in SP, in bytes, from the bottom
$05C0	return stack, 256 bytes	depth in RP
$06C0	compile stack, 256 bytes	depth in CSP; CFP marks the open definition's frame
$07C0	context stack, 64 bytes	depth in CXP; its bottom frame (interpreting) never goes
$0800	the dictionary	the kernel's words to KEND, then BOOT
$EC00	ROM: the trap messages	trap n points MSG at message n

Cells are 16 bits, little-endian (the 6502's order). **A word** is `PARENT(2) | LINK(2) LEN(1) NAME KIND(1) body`, its xt the address of LINK. KIND 0 is CODE (the body is one byte: the native number, an index into the jump table), 1 COLON (a thread), 2 CREATED (an ACTION cell, then data).

The kernel's own latches sit outside ITCH's 64K, as a CPU's internal state would: the WAIT latch, the host's stack pointer for a trap, and four addresses the kernel compiles itself (EXIT's xt, LIT's xt, the (UNSELF) thread, the seed colon's xt), which a ROM would have as link-time labels.

A trap (a stack underflow, UM/MOD OVERFLOW …) points MSG at the ROM message and unwinds to the host by restoring the saved stack pointer: a C longjmp, a 6502 BRK handler. Nothing is unwound in ITCH's RAM, so the stacks stay exactly as they were at the fault, for Forth to show.

Register use	65C02	x86-64	ARM64	RISC-V
RAM base	(absolute addressing)	rbx	x19	s0
IP	zero page $12 (ITCH's own)	r12	w20	s1
SP RP CSP CFP CXP …	zero page (ITCH's own)	in RAM	in RAM	in RAM
scratch	zero page $E0–$FC, A X Y	rax rcx rdx rsi rdi r8–r11	w0–w17	a0–a7 t0–t6
the kernel's latches	$BF00	ram + $10000	x22	s3

On the 6502 the fit is exact: **ITCH's register page is the 6502's zero page**. IP is used in place with `LDA (IP),Y`, and nothing is ever copied in or written back. On the 64-bit CPUs IP lives in a register (no Forth word reads $12), and every other register stays in RAM where Forth can read and write it (DEPTH is `12 @`, ABORT is `0 12 !`).

3. How the 2x2 design shaped this primitive set

ITCH has four stacks: data and return for running words, **compile** and **context** for building them. Several of its other choices follow from that. Together they are why the kernel needs only 41 primitives, why most of them are tiny, and why none of them needs an operating system.

- **No STATE: the context stack holds the mode.** COMPILING? is one indexed load; [and] are a push and a pop, so they nest, and] can never end a definition by accident. A traditional Forth's STATE variable, and every IF STATE @ in its compiler, disappear.
- **Control flow on its own stack.** IF and BEGIN leave their addresses on the compile stack (>C C>), not the data stack, and C> is bounds-checked against the open frame. That one check replaces fig-Forth's ?PAIRS tag numbers: BOOT's IF THEN ELSE BEGIN UNTIL are one line of Forth each, with no error handling, and an ELSE without IF is still caught.
- **A definition is a frame.** Four cells (outer frame, space, xt, hole) and a frame pointer, CFP. So ; can refuse to close a word with an IF still open, nested definitions come free (the hole is where the outer word jumps over the inner one), and :NONAME is the same path with an empty name. Two primitives, >CS and (CLOSE), open and close every kind of definition.

- **Where a word is built is data: the frame's space.** HERE , and PATCH are the only three words that write a definition, and they test the frame's space. So the host compiles into a second machine with its own unmodified IF THEN BEGIN LITERAL: cross-compilation, and the whole metacompiler, cost the kernel one test on three primitives.
- **Wordlists instead of flag bits.** A word acts while compiling because it lives in COMPILER, not because a bit is set in its header. IMMEDIATE and ALWAYS are Forth (they move a word between lists), and the interpreter never masks a flag.
- **DOES> knows SELF.** DODOES records the running data word, so DOES> code can make words that know their parent (ITCH's lineage). It costs one primitive, (UNSELF), and nothing for data words without DOES> code.
- **Devices are rings in RAM.** The terminal, keyboard and UART are byte rings with HEAD and TAIL registers. EMIT KEY SEND RECV are Forth; the kernel has no I/O primitives, only WAIT, which is the 65C02's WAI.
- **Errors are numbered traps.** A primitive that cannot go on points MSG at a ROM message. The kernel never builds a string, and Forth's own ERROR is just a store to MSG and HALT, itself -1 34 ! .
- **Registers in RAM at fixed addresses.** The machine's whole state is one byte array, so it can stop after any step and resume, be inspected, be read over a serial cord by another machine, and, as section 8 shows, run unchanged on four CPUs.

What is left: 9 inner-interpreter words, 6 arithmetic, 3 stack, 5 memory, 11 for the 2x2 compiler, 6 seed words that exist only because the host reads its language as text, and WAIT.

4. The inner interpreter

NEXT fetches the xt at IP and steps IP; EXECUTE finds the word's KIND byte (after its name) and dispatches: a CODE word through the jump table, a COLON word into DOCOL (push IP, IP := body), a CREATED word into DODOES (push its data address, and enter its DOES> code if it has any). After every step the loop checks RP (QUIT has returned), STOP (HALT) and the WAIT latch. These are the costs measured per step:

	65C02	x86-64	ARM64	RISC-V
DOCOL, whole step	61	34	40	42
DODOES, whole step	63	36	43	48
BRANCH, whole step (the dispatch plus a 2–8 instruction body)	47	30	36	41

65C02

```
itch_run:   NEXT until it stops -> A = 0 stopped, 1 asleep ( WAIT )

CSP = $02
IN = $04
NTIB = $06
LATEST = $08
HERE = $0A
SP = $0C
RP = $0E
IP = $12
SRC = $18
MSG = $1A
STOP = $22
SELF = $2A
CXP = $2C
CFP = $2E
COMMON = $30
COMPILER = $32
FORTH = $34
CURRENT = $36
PAD = $0440
DSB = $04C0
RSB = $05C0
CSB = $06C0
CXB = $07C0
ROM = $EC00
; the port's zero page ( in the TIB )
VAL = $E0
T1 = $E2
T2 = $E4
T3 = $E6
T4 = $E8
T5 = $EA
```

```
T6 = $EC
P1 = $EE
P2 = $F0
P3 = $F2
XT = $F4
CFA = $F6
ARG = $F8
N = $FA
F = $FB
ENTERED = $FC
; the kernel's own block: WAIT's latch, the host's stack pointer, the labels the kernel compiles
KB = $BF00
ASLEEP = KB
SAVES = KB+1
KEXIT = KB+$10
KLIT = KB+$12
KUNSELF = KB+$14
KSEED = KB+$16

; ── the four stack moves: \1 is a zero-page pair; A and X are used ──────────────────────────
.macro POPTO
        LDA SP
        SEC
        SBC #2
        TAX
        LDA SP+1
        SBC #0
        BCS ok\@
        JMP trap_ds_under
ok\@:
        STA SP+1
        STX SP
        LDA DSB,X
        STA \1
        LDA DSB+1,X
        STA \1+1
.endm
.macro PUSHFROM
        LDA SP+1
        BEQ ok\@
        JMP trap_ds_over
ok\@:
        LDX SP
        LDA \1
        STA DSB,X
        LDA \1+1
        STA DSB+1,X
        INX
        INX
        STX SP
        BNE done\@
        INC SP+1
done\@:
.endm
.macro RPOPTO
        LDA RP
        SEC
        SBC #2
        TAX
        LDA RP+1
        SBC #0
        BCS ok\@
        JMP trap_rs_under
ok\@:
        STA RP+1
        STX RP
        LDA RSB,X
        STA \1
        LDA RSB+1,X
        STA \1+1
.endm
.macro RPUSHFROM
        LDA RP+1
        BEQ ok\@
        JMP trap_rs_over
ok\@:
        LDX RP
        LDA \1
        STA RSB,X
        LDA \1+1
        STA RSB+1,X
        INX
        INX
        STX RP
        BNE done\@
```

```
        INC RP+1
done\@:
.endm
.macro VECTORED
        JSR frame_space
        LDA VAL
        ORA VAL+1
        BEQ \2
        LDY #\1
        LDA (VAL),Y
        STA XT
        INY
        LDA (VAL),Y
        STA XT+1
        JMP execute
.endm

        .org $C000
; ── entry points ──────────────────────────────────────────────────────────────────────────────────────────────────
itch_start:
        TSX
        STX SAVES
        STZ ENTERED
        LDA ARG
        STA XT
        LDA ARG+1
        STA XT+1
        JSR execute
        LDA ENTERED
        RTS
itch_run:
        TSX
        STX SAVES
run_loop:
        LDY #0                  ; NEXT: the xt at IP …
        LDA (IP),Y
        STA XT
        INY
        LDA (IP),Y
        STA XT+1
        CLC                     ; … IP past it …
        LDA IP
        ADC #2
        STA IP
        BCC nx1
        INC IP+1
nx1:
        JSR execute             ; … EXECUTE it
        LDA RP
        ORA RP+1
        BEQ run_stopped         ; its thread returned
        LDA STOP
        ORA STOP+1
        BNE run_stopped         ; HALT
        LDA ASLEEP
        BNE run_asleep          ; WAIT
        JMP run_loop
run_stopped:
        LDA #0
        RTS
run_asleep:
        LDA #1
        RTS

execute:
        LDY #2                  ; CFA = xt + 3 + LEN
        LDA (XT),Y
        AND #$7F
        CLC
        ADC #3
        ADC XT
        STA CFA
        LDA XT+1
        ADC #0
        STA CFA+1
        LDY #0
        LDA (CFA),Y             ; KIND
        BNE notcode
        INY
        LDA (CFA),Y             ; CODE: the native number …
        ASL A
        TAX
        JMP (natives,X)         ; … one indexed indirect jump ( its RTS returns to our caller )
notcode:
        CMP #2
```

```
        BEQ dodoes
docol:                          ; COLON: rpush(IP); IP := body
        LDA IP
        STA VAL
        LDA IP+1
        STA VAL+1
        RPUSHFROM VAL
        CLC
        LDA CFA
        ADC #1
        STA IP
        LDA CFA+1
        ADC #0
        STA IP+1
        LDA #1
        STA ENTERED
        RTS
dodoes:                         ; CREATED: push the data address; enter DOES> code if any
        CLC
        LDA CFA
        ADC #3
        STA VAL
        LDA CFA+1
        ADC #0
        STA VAL+1
        PUSHFROM VAL
        LDY #1
        LDA (CFA),Y
        STA T1
        INY
        LDA (CFA),Y
        STA T1+1
        ORA T1
        BEQ noact
        LDA IP
        STA VAL
        LDA IP+1
        STA VAL+1
        RPUSHFROM VAL           ; the way back …
        LDA SELF
        STA VAL
        LDA SELF+1
        STA VAL+1
        RPUSHFROM VAL           ; … the SELF it had …
        LDA KUNSELF
        STA VAL
        LDA KUNSELF+1
        STA VAL+1
        RPUSHFROM VAL           ; … and (UNSELF)
        LDA XT
        STA SELF
        LDA XT+1
        STA SELF+1
        LDA T1
        STA IP
        LDA T1+1
        STA IP+1
        LDA #1
        STA ENTERED
noact:
        RTS
```

x86-64

```
itch_run:                               # NEXT until RP is 0, STOP is set, WAIT sleeps, or the budget is spent
        push rbx
        push r12
        push r13
        push r14
        push r15
        mov rbx, rdi
        mov r15, rsi
        mov [rbx+K_RSP], rsp
        movzx r12d, word ptr [rbx+IP_]
run_loop:
        test r15, r15
        jz run_spent
        dec r15
        movzx edi, word ptr [rbx+r12]   # NEXT: the xt at IP …
        add r12d, 2                     # … IP steps past it …
        MASK r12d
        call execute                    # … and EXECUTE it
        cmp word ptr [rbx+RP_], 0       # its thread returned ( QUIT's EXIT )
```

```
        je run_stopped
        cmp word ptr [rbx+STOP_], 0     # HALT: Forth wrote the STOP register
        jne run_stopped
        cmp byte ptr [rbx+K_ASLEEP], 0  # WAIT: sleep until the frame
        jne run_asleep
        jmp run_loop
run_stopped:
        xor eax, eax
        jmp exit
run_asleep:
        mov eax, 1
        jmp exit
run_spent:
        mov eax, 2

execute:
        movzx eax, byte ptr [rbx+rdi+2] # LEN
        and eax, 127
        lea esi, [rdi+rax+3]            # CFA: the KIND byte
        MASK esi
        movzx eax, byte ptr [rbx+rsi]
        test eax, eax
        jnz 1f
        movzx eax, byte ptr [rbx+rsi+1] # CODE: the native number …
        lea rcx, [rip+natives]
        movsxd rdx, dword ptr [rcx+rax*4]
        add rdx, rcx
        call rdx                        # … through the jump table
        xor eax, eax
        ret
1:      cmp eax, 2
        je dodoes
docol:                                  # COLON: rpush(IP); IP := body
        mov eax, r12d
        RPUSH_AX
        lea r12d, [rsi+1]
        MASK r12d
        mov eax, 1
        ret
dodoes:                                 # CREATED: push the data address; enter DOES> code if any
        lea eax, [rsi+3]
        MASK eax
        PUSH_AX
        movzx edx, word ptr [rbx+rsi+1] # ACTION
        test edx, edx
        jz 2f
        mov eax, r12d
        RPUSH_AX                        # the way back …
        movzx eax, word ptr [rbx+SELF_]
        RPUSH_AX                        # … the SELF it had …
        movzx eax, word ptr [rbx+K_UNSELF]
        RPUSH_AX                        # … and (UNSELF), which restores both
        mov word ptr [rbx+SELF_], di
        mov r12d, edx
        mov eax, 1
        ret
2:      xor eax, eax
        ret

# ── the native jump table: the ORDER is the native number, as in the JS natives[] ──────────────────
        .p2align 2
```

ARM64

```
itch_run:
        stp x29, x30, [sp, #-48]!
        stp x19, x20, [sp, #16]
        stp x21, x22, [sp, #32]
        mov x19, x0
        mov x21, x1
        add x22, x19, #0x10, lsl #12
        mov x9, sp
        str x9, [x22, #8]
        ldrh w20, [x19, #IP_]
run_loop:
        cbz x21, run_spent
        sub x21, x21, #1
        add x10, x19, w20, uxtw         // NEXT: the xt at IP …
        ldrh w0, [x10]
        add w20, w20, #2                // … IP past it …
        and w20, w20, #0xFFFF
        bl execute                      // … EXECUTE it
        ldrh w9, [x19, #RP_]
```

```
        cbz w9, run_stopped             // its thread returned
        ldrh w9, [x19, #STOP_]
        cbnz w9, run_stopped            // HALT
        ldrb w9, [x22]
        cbnz w9, run_asleep             // WAIT
        b run_loop
run_stopped:
        mov w0, #0
        b exit
run_asleep:
        mov w0, #1
        b exit
run_spent:
        mov w0, #2

execute:
        ENTER
        and w0, w0, #0xFFFF
        add x10, x19, w0, uxtw
        ldrb w9, [x10, #2]              // LEN
        and w9, w9, #127
        add w1, w0, w9
        add w1, w1, #3
        and w1, w1, #0xFFFF             // CFA: the KIND byte
        add x11, x19, w1, uxtw
        ldrb w9, [x11]
        cbnz w9, 1f
        ldrb w9, [x11, #1]              // CODE: the native number, through the jump table
        adr x12, natives
        ldrsw x13, [x12, w9, uxtw #2]
        add x12, x12, x13
        blr x12
        mov w0, #0
        LEAVE
1:      cmp w9, #2
        b.eq dodoes
docol:                                  // COLON: rpush(IP); IP := body
        mov w0, w20
        RPUSH_W0
        add w20, w1, #1
        and w20, w20, #0xFFFF
        mov w0, #1
        LEAVE
dodoes:                                 // CREATED: push the data address; enter DOES> code if any
        mov w3, w0
        add w0, w1, #3
        and w0, w0, #0xFFFF
        PUSH_W0
        add x11, x19, w1, uxtw
        ldurh w4, [x11, #1]             // ACTION ( an odd address: ARM64 loads it unaligned )
        cbz w4, 2f
        mov w0, w20
        RPUSH_W0
        ldrh w0, [x19, #SELF_]
        RPUSH_W0
        ldrh w0, [x22, #20]             // UNSELF_CODE
        RPUSH_W0
        strh w3, [x19, #SELF_]
        mov w20, w4
        mov w0, #1
        LEAVE
2:      mov w0, #0
        LEAVE

        .p2align 2
```

RISC-V (RV64IM)

```
itch_run:
        addi sp, sp, -48
        sd ra, 40(sp)
        sd s0, 32(sp)
        sd s1, 24(sp)
        sd s2, 16(sp)
        sd s3, 8(sp)
        sd s4, 0(sp)
        mv s0, a0
        mv s2, a1
        li t0, 0x10000
        add s3, s0, t0
        li s4, 0xFFFF
        sd sp, 8(s3)
        lhu s1, IP_(s0)
```

```
run_loop:
        beqz s2, run_spent
        addi s2, s2, -1
        add t0, s0, s1                  # NEXT: the xt at IP …
        lhu a0, 0(t0)
        addi s1, s1, 2                  # … IP past it …
        and s1, s1, s4
        jal ra, execute                 # … EXECUTE it
        lhu t0, RP_(s0)
        beqz t0, run_stopped            # its thread returned
        lhu t0, STOP_(s0)
        bnez t0, run_stopped            # HALT
        lbu t0, 0(s3)
        bnez t0, run_asleep             # WAIT
        j run_loop
run_stopped:
        li a0, 0
        j exit
run_asleep:
        li a0, 1
        j exit
run_spent:
        li a0, 2

execute:
        ENTER
        and a0, a0, s4
        add t0, s0, a0
        lbu t1, 2(t0)                   # LEN
        andi t1, t1, 127
        add a1, a0, t1
        addi a1, a1, 3
        and a1, a1, s4                  # CFA: the KIND byte
        add t0, s0, a1
        lbu t1, 0(t0)
        bnez t1, 1f
        lbu t1, 1(t0)                   # CODE: the native number, through the jump table
        slli t1, t1, 2
        lla t2, natives                 # auipc + addi, PC-relative
        add t1, t2, t1
        lw t1, 0(t1)
        add t1, t2, t1
        jalr ra, 0(t1)
        li a0, 0
        LEAVE
1:      li t2, 2
        beq t1, t2, dodoes
docol:                                  # COLON: rpush(IP); IP := body
        mv a0, s1
        RPUSH_A0
        addi s1, a1, 1
        and s1, s1, s4
        li a0, 1
        LEAVE
dodoes:                                 # CREATED: push the data address; enter DOES> code if any
        mv a3, a0
        addi a0, a1, 3
        and a0, a0, s4
        PUSH_A0
        add t0, s0, a1
        lbu a4, 1(t0)                   # ACTION, byte by byte ( it may sit at an odd address )
        lbu t1, 2(t0)
        slli t1, t1, 8
        or a4, a4, t1
        beqz a4, 2f
        mv a0, s1
        RPUSH_A0
        lhu a0, SELF_(s0)
        RPUSH_A0
        lhu a0, 20(s3)                  # UNSELF_CODE
        RPUSH_A0
        sh a3, SELF_(s0)
        mv s1, a4
        li a0, 1
        LEAVE
2:      li a0, 0
        LEAVE

        .p2align 2
```

5. The stack macros

Most primitives are built from four moves, each checking its bound in the JavaScript kernel's order: push and pop on the data stack and on the return stack. They are macros, so each use is inline code. The same files also define VECTORED (the test that sends HERE , PATCH to another space) and, on the RISC CPUs, the frame save around calls.

65C02

```
.macro POPTO
        LDA SP
        SEC
        SBC #2
        TAX
        LDA SP+1
        SBC #0
        BCS ok\@
        JMP trap_ds_under
ok\@:
        STA SP+1
        STX SP
        LDA DSB,X
        STA \1
        LDA DSB+1,X
        STA \1+1
.endm

.macro PUSHFROM
        LDA SP+1
        BEQ ok\@
        JMP trap_ds_over
ok\@:
        LDX SP
        LDA \1
        STA DSB,X
        LDA \1+1
        STA DSB+1,X
        INX
        INX
        STX SP
        BNE done\@
        INC SP+1
done\@:
.endm

.macro RPOPTO
        LDA RP
        SEC
        SBC #2
        TAX
        LDA RP+1
        SBC #0
        BCS ok\@
        JMP trap_rs_under
ok\@:
        STA RP+1
        STX RP
        LDA RSB,X
        STA \1
        LDA RSB+1,X
        STA \1+1
.endm

.macro RPUSHFROM
        LDA RP+1
        BEQ ok\@
        JMP trap_rs_over
ok\@:
        LDX RP
        LDA \1
        STA RSB,X
        LDA \1+1
        STA RSB+1,X
        INX
        INX
        STX RP
        BNE done\@
        INC RP+1
done\@:
.endm

.macro VECTORED
```

```
        JSR frame_space
        LDA VAL
        ORA VAL+1
        BEQ \2
        LDY #\1
        LDA (VAL),Y
        STA XT
        INY
        LDA (VAL),Y
        STA XT+1
        JMP execute
.endm
```

x86-64

```
.macro PUSH_AX                          # push(ax): overflow if SP >= 256
        movzx ecx, word ptr [rbx+SP_]
        cmp ecx, 256
        jae trap_ds_over
        mov word ptr [rbx+rcx+DSB], ax
        add ecx, 2
        mov word ptr [rbx+SP_], cx
.endm

.macro POP_AX                           # eax := pop(), sign-extended: underflow if SP < 2
        movzx ecx, word ptr [rbx+SP_]
        sub ecx, 2
        jb trap_ds_under
        mov word ptr [rbx+SP_], cx
        movsx eax, word ptr [rbx+rcx+DSB]
.endm

.macro RPUSH_AX
        movzx ecx, word ptr [rbx+RP_]
        cmp ecx, 256
        jae trap_rs_over
        mov word ptr [rbx+rcx+RSB], ax
        add ecx, 2
        mov word ptr [rbx+RP_], cx
.endm

.macro RPOP_AX                          # eax := rpop(), zero-extended
        movzx ecx, word ptr [rbx+RP_]
        sub ecx, 2
        jb trap_rs_under
        mov word ptr [rbx+RP_], cx
        movzx eax, word ptr [rbx+rcx+RSB]
.endm

.macro MASK r                           # an ITCH address is 16 bits
        and \r, 0xFFFF
.endm

.macro VECTORED slot, local
        call frame_space
        test eax, eax
        jz \local
        movzx edi, word ptr [rbx+rax+\slot]
        jmp execute                     # enter it: DOCOL, and its EXIT returns to our caller's thread
.endm
```

ARM64

```
.macro PUSH_W0
        ldrh w9, [x19, #SP_]
        cmp w9, #256
        b.hs trap_ds_over
        add x10, x19, w9, uxtw
        strh w0, [x10, #DSB]
        add w9, w9, #2
        strh w9, [x19, #SP_]
.endm

.macro POP_W0                           // w0 := pop(), signed
        ldrh w9, [x19, #SP_]
        subs w9, w9, #2
        b.lo trap_ds_under
        strh w9, [x19, #SP_]
        add x10, x19, w9, uxtw
        ldrsh w0, [x10, #DSB]
.endm

.macro RPUSH_W0
        ldrh w9, [x19, #RP_]
        cmp w9, #256
        b.hs trap_rs_over
        add x10, x19, w9, uxtw
        strh w0, [x10, #RSB]
        add w9, w9, #2
        strh w9, [x19, #RP_]
.endm

.macro RPOP_W0                          // w0 := rpop(), unsigned
        ldrh w9, [x19, #RP_]
        subs w9, w9, #2
        b.lo trap_rs_under
        strh w9, [x19, #RP_]
        add x10, x19, w9, uxtw
        ldrh w0, [x10, #RSB]
.endm

.macro ENTER                            // a routine that calls others keeps its link
        stp x29, x30, [sp, #-16]!
.endm

.macro LEAVE
        ldp x29, x30, [sp], #16
        ret
.endm

.macro VECTORED slot, local
        ENTER
        bl frame_space
        LEAVE_KEEP
        cbz w0, \local
        add x10, x19, w0, uxtw
        ldrh w0, [x10, #\slot]
        b execute
.endm

.macro LEAVE_KEEP
        ldp x29, x30, [sp], #16
.endm
```

RISC-V (RV64IM)

```
.macro PUSH_A0
        lhu t0, SP_(s0)
        li t1, 256
        bltu t0, t1, .Lok\@
        j trap_ds_over
.Lok\@: add t2, s0, t0
        sh a0, DSB(t2)
        addi t0, t0, 2
        sh t0, SP_(s0)
.endm

.macro POP_A0                           # a0 := pop(), signed
        lhu t0, SP_(s0)
        addi t0, t0, -2
        bgez t0, .Lok\@
        j trap_ds_under
.Lok\@: sh t0, SP_(s0)
```

```
        add t2, s0, t0
        lh a0, DSB(t2)
.endm

.macro RPUSH_A0
        lhu t0, RP_(s0)
        li t1, 256
        bltu t0, t1, .Lok\@
        j trap_rs_over
.Lok\@: add t2, s0, t0
        sh a0, RSB(t2)
        addi t0, t0, 2
        sh t0, RP_(s0)
.endm

.macro RPOP_A0                          # a0 := rpop(), unsigned
        lhu t0, RP_(s0)
        addi t0, t0, -2
        bgez t0, .Lok\@
        j trap_rs_under
.Lok\@: sh t0, RP_(s0)
        add t2, s0, t0
        lhu a0, RSB(t2)
.endm

.macro ENTER
        addi sp, sp, -16
        sd ra, 8(sp)
.endm

.macro LEAVE
        ld ra, 8(sp)
        addi sp, sp, 16
        ret
.endm

.macro JNZ r, target                    # a far conditional jump ( to a trap )
        beqz \r, .Lz\@
        j \target
.Lz\@:
.endm

.macro JZ r, target
        bnez \r, .Lz\@
        j \target
.Lz\@:
.endm

.macro VECTORED slot, local
        ENTER
        jal ra, frame_space
        ld ra, 8(sp)
        addi sp, sp, 16
        beqz a0, \local
        add t2, s0, a0
        lhu a0, \slot(t2)
        j execute
.endm
```

6. The 41 primitives

For each: its stack effect, what it does, why it is a primitive and what ITCH's design does to it; then its cost (average instructions per call, from entry to return, including the helpers it calls and every check, measured over the proof workload) and its code on all four CPUs, cut from the proven sources.

The inner interpreter's own words

These nine are the virtual machine itself: any threaded Forth has them in some form, because Forth cannot say "jump", "push the next cell" or "return" in Forth. They touch IP and the return stack, which no colon word may do directly.

`LIT` (-- x)

Push the cell that follows in the thread, and step IP past it. Numbers in a definition are compiled as LIT n. Only the inner interpreter knows where IP points.

Native #22 · called 262,886 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	23	10	12	12
6502 cycles per call	74			

65C02

```
p_lit:
        LDY #0
        LDA (IP),Y
        STA VAL
        INY
        LDA (IP),Y
        STA VAL+1
        PUSHFROM VAL
        CLC
        LDA IP
        ADC #2
        STA IP
        BCC lit1
        INC IP+1
lit1:
        RTS
```

x86-64

```
p_lit:                                  # LIT  ( -- x ): the inline cell, then step past it
        movzx eax, word ptr [rbx+r12]
        PUSH_AX
        add r12d, 2
        MASK r12d
        ret
```

ARM64

```
p_lit:
        add x11, x19, w20, uxtw
        ldrh w0, [x11]
        PUSH_W0
        add w20, w20, #2
        and w20, w20, #0xFFFF
        ret
```

RISC-V (RV64IM)

```
p_lit:
        add t3, s0, s1
        lhu a0, 0(t3)
        PUSH_A0
        addi s1, s1, 2
        and s1, s1, s4
        ret
```

`EXIT` (--)

Return from a colon word: IP := rpop(). The other half of DOCOL. ITCH's ; lays it, and it is the most executed word of all (288,782 times in the proof run).

Native #23 · called 288,783 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	14	7	8	8
6502 cycles per call	43			

65C02

```
p_exit:
        RPOPTO IP
        RTS
```

x86-64

```
p_exit:                             # EXIT: back to the caller
        RPOP_AX
        mov r12d, eax
        ret
```

ARM64

```
p_exit:
        RPOP_W0
        mov w20, w0
        ret
```

RISC-V (RV64IM)

```
p_exit:
        RPOP_A0
        mv s1, a0
        ret
```

`BRANCH` (--)

IP := the inline cell. IF ELSE BEGIN AGAIN REPEAT compile it. In ITCH it also lets a nested definition be jumped over: the outer word gets BRANCH <hole>, and the inner ; fills the hole.

Native #24 · called 117,485 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	8	2	3	3
6502 cycles per call	28			

65C02

```
p_branch:
        LDY #0
        LDA (IP),Y
        TAX
        INY
        LDA (IP),Y
        STA IP+1
        STX IP
        RTS
```

x86-64

```
p_branch:                           # BRANCH: IP := the inline cell
        movzx r12d, word ptr [rbx+r12]
        ret
```

ARM64

```
p_branch:
        add x11, x19, w20, uxtw
        ldrh w20, [x11]
        ret
```

RISC-V (RV64IM)

```
p_branch:
        add t3, s0, s1
        lhu s1, 0(t3)
        ret
```

`0BRANCH` (f --)

Jump to the inline cell when f is 0, else step over it. The only conditional in the machine: IF WHILE UNTIL compile it.

Native #25 · called 222,611 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	23	10	10	10
6502 cycles per call	68			

65C02

```
p_zbranch:
        POPTO VAL
        LDA VAL
        ORA VAL+1
        BEQ p_branch
        CLC
        LDA IP
        ADC #2
        STA IP
        BCC zb1
        INC IP+1
zb1:
        RTS
```

x86-64

```
p_zbranch:                              # 0BRANCH  ( f -- ): jump when f is 0
        POP_AX
        test eax, eax
        jz 1f
        add r12d, 2
        MASK r12d
        ret
1:      movzx r12d, word ptr [rbx+r12]
        ret
```

ARM64

```
p_zbranch:
        POP_W0
        cbz w0, 1f
        add w20, w20, #2
        and w20, w20, #0xFFFF
        ret
1:      add x11, x19, w20, uxtw
        ldrh w20, [x11]
        ret
```

RISC-V (RV64IM)

```
p_zbranch:
        POP_A0
        beqz a0, 1f
        addi s1, s1, 2
        and s1, s1, s4
        ret
1:      add t3, s0, s1
        lhu s1, 0(t3)
        ret
```

`EXECUTE` (xt --)

Run the word xt: the machine's dispatcher, on the stack. Forth needs to run a word it holds as a value. ITCH's outer interpreter runs every word it finds this way, so EXECUTE's per-call cost includes the word it enters.

Native #18 · called 1,477 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	96	48	53	56
6502 cycles per call	285			

65C02

```
p_execute:
        POPTO XT
        JMP execute
```

x86-64

```
p_execute:                              # EXECUTE  ( xt -- )
        POP_AX
        MASK eax
        mov edi, eax
        jmp execute
```

ARM64

```
p_execute:
        POP_W0
        b execute
```

RISC-V (RV64IM)

```
p_execute:
        POP_A0
        j execute
```

`>R` (x --) (R: -- x)

Move a cell to the return stack. Colon words may borrow the return stack, but only the kernel can reach it. Bounds-checked: RETURN STACK OVERFLOW is a trap, not a crash.

Native #19 · called 59,558 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	25	12	14	14
6502 cycles per call	78			

65C02

```
p_tor:
        POPTO VAL
        RPUSHFROM VAL
        RTS
```

x86-64

```
p_tor:                                  # >R  ( x -- ) ( R: -- x )
        POP_AX
        RPUSH_AX
        ret
```

ARM64

```
p_tor:
        POP_W0
        RPUSH_W0
        ret
```

RISC-V (RV64IM)

```
p_tor:
        POP_A0
        RPUSH_A0
        ret
```

`R>` (-- x) (R: x --)

Move a cell back from the return stack. The pair of >R. R@ is Forth (R> R> DUP >R SWAP >R), so it needs no primitive.

Native #20 · called 59,559 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	25	12	14	14
6502 cycles per call	78			

65C02

```
p_rfrom:
        RPOPTO VAL
        PUSHFROM VAL
        RTS
```

x86-64

```
p_rfrom:                                # R>  ( -- x ) ( R: x -- )
        RPOP_AX
        PUSH_AX
        ret
```

ARM64

```
p_rfrom:
        RPOP_W0
        PUSH_W0
        ret
```

RISC-V (RV64IM)

```
p_rfrom:
        RPOP_A0
        PUSH_A0
        ret
```

`RP!` (n --)

Set the return stack's depth (bytes). The one way Forth can cut the return stack back: ITCH's registers live in RAM at $0E, so this is a single store.

Native #21 · called 1 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	18	7	8	8
6502 cycles per call	55			

65C02

```
p_rpstore:
        POPTO VAL
        LDA VAL
        STA RP
        LDA VAL+1
        STA RP+1
        RTS
```

x86-64

```
p_rpstore:                              # RP!  ( n -- ): the return stack's depth
        POP_AX
        mov word ptr [rbx+RP_], ax
        ret
```

ARM64

```
p_rpstore:
        POP_W0
        strh w0, [x19, #RP_]
        ret
```

RISC-V (RV64IM)

```
p_rpstore:
        POP_A0
        sh a0, RP_(s0)
        ret
```

`(UNSELF)` (--) (R: ip self --)

Where DOES> code returns: restore SELF, then IP. ITCH's DODOES remembers the running data word in SELF ($2A) so that DOES> code can make children with a PARENT. DODOES pushes IP, the old SELF and the address of a one-cell thread holding (UNSELF); when the DOES> code EXITs into that thread, this puts both back. A data word without DOES> code costs nothing: DODOES only pushes when there is code to enter.

Native #35 · called 311 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	27	13	15	15
6502 cycles per call	80			

65C02

```
p_unself:
        RPOPTO SELF
        RPOPTO IP
        RTS
```

x86-64

```
p_unself:                               # (UNSELF): where DOES> code returns: SELF back, then return
        RPOP_AX
        mov word ptr [rbx+SELF_], ax
        RPOP_AX
        mov r12d, eax
        ret
```

ARM64

```
p_unself:
        RPOP_W0
        strh w0, [x19, #SELF_]
        RPOP_W0
        mov w20, w0
        ret
```

RISC-V (RV64IM)

```
p_unself:
        RPOP_A0
        sh a0, SELF_(s0)
        RPOP_A0
        mv s1, a0
        ret
```

Arithmetic and logic

Everything else in arithmetic is Forth in BOOT: NEGATE INVERT OR XOR 1+ 1- MOD < U< 0= … are built from these six. On a CPU without MUL or DIV (the 6502) three of them are loops.

`+` (a b -- a+b)

Add, 16 bits, wrapping. The one addition everything else is built from.

Native #0 · called 52,861 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	45	19	22	22
6502 cycles per call	135			

65C02

```
p_plus:
        POPTO T1
        POPTO VAL
        CLC
        LDA VAL
        ADC T1
        STA VAL
        LDA VAL+1
        ADC T1+1
        STA VAL+1
        PUSHFROM VAL
        RTS
```

x86-64

```
p_plus:                                 # +  ( a b -- a+b )
        POP_AX
        mov edx, eax
        POP_AX
        add eax, edx
        PUSH_AX
        ret
```

ARM64

```
p_plus:
        POP_W0
        mov w2, w0
        POP_W0
        add w0, w0, w2
        PUSH_W0
        ret
```

RISC-V (RV64IM)

```
p_plus:
        POP_A0
        mv a1, a0
        POP_A0
        add a0, a0, a1
        PUSH_A0
        ret
```

`-` (a b -- a-b)

Subtract, 16 bits, wrapping. With + and AND it gives NEGATE, INVERT and OR in BOOT's first lines.

Native #1 · called 79,518 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	45	19	22	22
6502 cycles per call	135			

65C02

```
p_minus:
        POPTO T1
        POPTO VAL
        SEC
        LDA VAL
        SBC T1
        STA VAL
        LDA VAL+1
        SBC T1+1
        STA VAL+1
        PUSHFROM VAL
        RTS
```

x86-64

```
p_minus:                                # -  ( a b -- a-b )
        POP_AX
        mov edx, eax
        POP_AX
        sub eax, edx
        PUSH_AX
        ret
```

ARM64

```
p_minus:
        POP_W0
        mov w2, w0
        POP_W0
        sub w0, w0, w2
        PUSH_W0
        ret
```

RISC-V (RV64IM)

```
p_minus:
        POP_A0
        mv a1, a0
        POP_A0
        sub a0, a0, a1
        PUSH_A0
        ret
```

`*` (a b -- a*b)

Multiply: the low 16 bits. Too slow in Forth. On the 6502 it is a 16-step shift-and-add loop; on the others one instruction.

Native #2 · called 396 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	162	19	22	22
6502 cycles per call	595			

65C02

```
p_star:                         ; the low 16 bits: shift and add
        POPTO T1
        POPTO T2
        STZ VAL
        STZ VAL+1
        LDX #16
mul1:
        LSR T1+1
        ROR T1
        BCC mul2
        CLC
        LDA VAL
        ADC T2
        STA VAL
        LDA VAL+1
        ADC T2+1
        STA VAL+1
mul2:
        ASL T2
        ROL T2+1
        DEX
        BNE mul1
        PUSHFROM VAL
        RTS
```

x86-64

```
p_star:                                 # *  ( a b -- a*b ): the low 16 bits
        POP_AX
        mov edx, eax
        POP_AX
        imul eax, edx
        PUSH_AX
        ret
```

ARM64

```
p_star:
        POP_W0
        mov w2, w0
        POP_W0
        mul w0, w0, w2
        PUSH_W0
        ret
```

RISC-V (RV64IM)

```
p_star:
        POP_A0
        mv a1, a0
        POP_A0
        mulw a0, a0, a1
        PUSH_A0
        ret
```

`/` (a b -- q)

Signed divide, toward zero. A zero divisor gives 0 (the JavaScript kernel's rule); -32768 / -1 gives -32768 ($8000). ARM64's SDIV already gives 0 for a zero divisor; x86 would fault and RISC-V gives -1, so those two test for it. The 6502 has no divide: magnitudes, a 16-step loop, then the sign.

Native #3 · called 4 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	220	22	22	23
6502 cycles per call	722			

65C02

```
p_slash:                        ; toward zero; a zero divisor gives 0
        POPTO T1
        POPTO T2
        LDA T1
        ORA T1+1
        BNE div1
        STZ VAL
        STZ VAL+1
        BRA divpush
div1:
        LDA T2+1
        EOR T1+1
        STA F                   ; the quotient's sign
        LDA T2+1
        BPL div2
        SEC
        LDA #0
        SBC T2
        STA T2
        LDA #0
        SBC T2+1
        STA T2+1
div2:
        LDA T1+1
        BPL div3
        SEC
        LDA #0
        SBC T1
        STA T1
        LDA #0
        SBC T1+1
        STA T1+1
div3:
        JSR udiv16
        LDA F
        BPL divpush
        SEC
        LDA #0
        SBC VAL
        STA VAL
        LDA #0
        SBC VAL+1
        STA VAL+1
divpush:
        PUSHFROM VAL
        RTS
```

x86-64

```
p_slash:                                # /  ( a b -- q ): toward zero; b = 0 gives 0 ( as the JS does )
        POP_AX
        mov r8d, eax
        POP_AX
        test r8d, r8d
        jz 1f
        cdq
        idiv r8d                        # 32-bit: -32768 / -1 is 32768, stored as $8000
        PUSH_AX
        ret
1:      xor eax, eax
        PUSH_AX
        ret
```

ARM64

```
p_slash:                                // ARM64's SDIV gives 0 for a zero divisor: exactly ITCH's rule
        POP_W0
        mov w2, w0
        POP_W0
        sdiv w0, w0, w2
        PUSH_W0
        ret
```

RISC-V (RV64IM)

```
p_slash:                                # a zero divisor gives 0 ( DIVW would give -1 )
        POP_A0
        mv a1, a0
        POP_A0
        beqz a1, 1f
        divw a0, a0, a1
        PUSH_A0
        ret
1:      li a0, 0
        PUSH_A0
        ret
```

`AND` (a b -- a&b)

Bitwise AND. With - it builds INVERT and OR: BOOT's OR is INVERT SWAP INVERT AND INVERT.

Native #12 · called 71,823 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	44	19	22	22
6502 cycles per call	133			

65C02

```
p_and:
        POPTO T1
        POPTO VAL
        LDA VAL
        AND T1
        STA VAL
        LDA VAL+1
        AND T1+1
        STA VAL+1
        PUSHFROM VAL
        RTS
```

x86-64

```
p_and:                                  # AND  ( a b -- a&b )
        POP_AX
        mov edx, eax
        POP_AX
        and eax, edx
        PUSH_AX
        ret
```

ARM64

```
p_and:
        POP_W0
        mov w2, w0
        POP_W0
        and w0, w0, w2
        PUSH_W0
        ret
```

RISC-V (RV64IM)

```
p_and:
        POP_A0
        mv a1, a0
        POP_A0
        and a0, a0, a1
        PUSH_A0
        ret
```

`UM/MOD` (lo hi u -- rem quot)

Divide the unsigned 32-bit number hi:lo by u. Traps with UM/MOD OVERFLOW when hi >= u (the quotient would not fit, u = 0 included). Number output (U. and . in the NUMBERS module) needs a 32-by-16 divide. It is the one double-cell operation in the kernel.

Native #14 · called 71 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	298	40	42	42
6502 cycles per call	954			

65C02

```
p_ummod:                        ; UM/MOD ( lo hi u -- rem quot ): 32 by 16 bits, shift and subtract
        POPTO T1                ; u
        POPTO T2                ; hi
        POPTO T3                ; lo
        LDA T2+1                ; hi >= u: trap
        CMP T1+1
        BCC um0
        BNE um_bad
        LDA T2
        CMP T1
        BCC um0
um_bad:
        JMP trap_ummod
um0:
        LDX #16
um1:
        ASL T3
        ROL T3+1
        ROL T2
        ROL T2+1
        BCS um2                 ; a 17th bit: certainly >= u
        SEC
        LDA T2
        SBC T1
        TAY
        LDA T2+1
        SBC T1+1
        BCC um3
        STA T2+1
        STY T2
        INC T3
        BRA um3
um2:
        LDA T2
        SBC T1
        STA T2
        LDA T2+1
        SBC T1+1
        STA T2+1
        INC T3
um3:
        DEX
        BNE um1
        PUSHFROM T2             ; the remainder
        PUSHFROM T3             ; the quotient
        RTS
```

x86-64

```
p_ummod:                                # UM/MOD  ( lo hi u -- rem quot )
        POP_AX
        movzx r8d, ax                   # u
        POP_AX
        movzx edx, ax                   # hi
        POP_AX
        movzx eax, ax                   # lo
        cmp edx, r8d                    # hi >= u: the quotient would not fit ( u = 0 included )
        jae trap_ummod
        shl edx, 16
        or eax, edx
        xor edx, edx
        div r8d
        mov r9d, eax
        mov eax, edx
        PUSH_AX
        mov eax, r9d
        PUSH_AX
        ret
```

ARM64

```
p_ummod:                                // UM/MOD ( lo hi u -- rem quot )
        POP_W0
        and w2, w0, #0xFFFF             // u
        POP_W0
        and w3, w0, #0xFFFF             // hi
        POP_W0
        and w4, w0, #0xFFFF             // lo
        cmp w3, w2
        b.hs trap_ummod
        orr w4, w4, w3, lsl #16
        udiv w5, w4, w2
        msub w0, w5, w2, w4             // the remainder
        PUSH_W0
        mov w0, w5
        PUSH_W0
        ret
```

RISC-V (RV64IM)

```
p_ummod:
        POP_A0
        and a1, a0, s4                  # u
        POP_A0
        and a2, a0, s4                  # hi
        POP_A0
        and a3, a0, s4                  # lo
        bltu a2, a1, 1f
        j trap_ummod
1:      slli a2, a2, 16
        or a3, a3, a2
        divuw a4, a3, a1
        remuw a0, a3, a1
        PUSH_A0
        mv a0, a4
        PUSH_A0
        ret
```

The stack

Only three: OVER ROT NIP 2DUP 2DROP R@ are Forth in BOOT (each tagged [PRIM], so a port can make it code). Each move is checked: a pop from an empty stack is the trap DATA STACK UNDERFLOW with the stacks left as they were, which is why DUP is written as a pop and two pushes, exactly in the JavaScript kernel's order.

`DUP` (x -- x x)

Copy the top cell. A stack primitive: Forth cannot copy a cell without one.

Native #4 · called 146,013 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	36	20	23	23
6502 cycles per call	113			

65C02

```
p_dup:
        POPTO VAL
        PUSHFROM VAL
        PUSHFROM VAL
        RTS
```

x86-64

```
p_dup:                                  # DUP  ( x -- x x )
        POP_AX
        mov edx, eax
        PUSH_AX
        mov eax, edx
        PUSH_AX
        ret
```

ARM64

```
p_dup:
        POP_W0
        mov w2, w0
        PUSH_W0
        mov w0, w2
        PUSH_W0
        ret
```

RISC-V (RV64IM)

```
p_dup:
        POP_A0
        mv a1, a0
        PUSH_A0
        mv a0, a1
        PUSH_A0
        ret
```

`DROP` (x --)

Discard the top cell. The cheapest pop.

Native #5 · called 16,130 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	14	6	7	7
6502 cycles per call	43			

65C02

```
p_drop:
        POPTO VAL
        RTS
```

x86-64

```
p_drop:                                 # DROP  ( x -- )
        POP_AX
        ret
```

ARM64

```
p_drop:
        POP_W0
        ret
```

RISC-V (RV64IM)

```
p_drop:
        POP_A0
        ret
```

`SWAP` (a b -- b a)

Exchange the top two cells. With DUP DROP >R R> it builds every other stack word in BOOT.

Native #6 · called 70,035 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	49	27	31	31
6502 cycles per call	150			

65C02

```
p_swap:
        POPTO T1
        POPTO VAL
        PUSHFROM T1
        PUSHFROM VAL
        RTS
```

x86-64

```
p_swap:                                 # SWAP  ( a b -- b a )
        POP_AX
        mov edx, eax
        POP_AX
        mov r8d, eax
        mov eax, edx
        PUSH_AX
        mov eax, r8d
        PUSH_AX
        ret
```

ARM64

```
p_swap:
        POP_W0
        mov w2, w0
        POP_W0
        mov w3, w0
        mov w0, w2
        PUSH_W0
        mov w0, w3
        PUSH_W0
        ret
```

RISC-V (RV64IM)

```
p_swap:
        POP_A0
        mv a1, a0
        POP_A0
        mv a2, a0
        mv a0, a1
        PUSH_A0
        mv a0, a2
        PUSH_A0
        ret
```

Memory

The whole machine is one 64K byte array, and Forth reaches every part of it through these five: the dictionary, the stacks' registers, the devices' rings. There are no I/O primitives at all: EMIT, KEY, SEND and RECV are Forth reading and writing the chips' registers with C@ and C!.

`@` (a -- x)

Fetch the cell at a (little-endian, signed). Every register (SP at $0C, HERE at $0A …) and every device is read this way.

Native #7 · called 92,768 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	31	14	17	17
6502 cycles per call	98			

65C02

```
p_fetch:
        POPTO P1
        LDY #0
        LDA (P1),Y
        STA VAL
        INY
        LDA (P1),Y
        STA VAL+1
        PUSHFROM VAL
        RTS
```

x86-64

```
p_fetch:                                # @  ( a -- x )
        POP_AX
        MASK eax
        movzx eax, word ptr [rbx+rax]
        PUSH_AX
        ret
```

ARM64

```
p_fetch:
        POP_W0
        and w0, w0, #0xFFFF
        add x11, x19, w0, uxtw
        ldrh w0, [x11]
        PUSH_W0
        ret
```

RISC-V (RV64IM)

```
p_fetch:
        POP_A0
        and a0, a0, s4
        add t3, s0, a0
        lhu a0, 0(t3)
        PUSH_A0
        ret
```

`!` (x a --)

Store x at a. HALT is -1 34 ! : Forth stops the machine by writing the STOP register.

Native #8 · called 2,351 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	33	14	16	16
6502 cycles per call	102			

65C02

```
p_store:
        POPTO P1
        POPTO VAL
        LDY #0
        LDA VAL
        STA (P1),Y
        INY
        LDA VAL+1
        STA (P1),Y
        RTS
```

x86-64

```
p_store:                                # !  ( x a -- )
        POP_AX
        MASK eax
        mov edx, eax
        POP_AX
        mov word ptr [rbx+rdx], ax
        ret
```

ARM64

```
p_store:
        POP_W0
        and w2, w0, #0xFFFF
        POP_W0
        add x11, x19, w2, uxtw
        strh w0, [x11]
        ret
```

RISC-V (RV64IM)

```
p_store:
        POP_A0
        and a1, a0, s4
        POP_A0
        add t3, s0, a1
        sh a0, 0(t3)
        ret
```

`C@` (a -- c)

Fetch a byte. The rings of the terminal, keyboard and UART have byte-sized HEAD and TAIL, so they wrap by themselves.

Native #9 · called 53,548 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	28	14	17	17
6502 cycles per call	90			

65C02

```
p_cfetch:
        POPTO P1
        LDA (P1)
        STA VAL
        STZ VAL+1
        PUSHFROM VAL
        RTS
```

x86-64

```
p_cfetch:                               # C@  ( a -- c )
        POP_AX
        MASK eax
        movzx eax, byte ptr [rbx+rax]
        PUSH_AX
        ret
```

ARM64

```
p_cfetch:
        POP_W0
        and w0, w0, #0xFFFF
        add x11, x19, w0, uxtw
        ldrb w0, [x11]
        PUSH_W0
        ret
```

RISC-V (RV64IM)

```
p_cfetch:
        POP_A0
        and a0, a0, s4
        add t3, s0, a0
        lbu a0, 0(t3)
        PUSH_A0
        ret
```

`C!` (c a --)

Store a byte. EMIT is a C! into the terminal chip's ring.

Native #10 · called 1,821 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	29	14	16	16
6502 cycles per call	88			

65C02

```
p_cstore:
        POPTO P1
        POPTO VAL
        LDA VAL
        STA (P1)
        RTS
```

x86-64

```
p_cstore:                               # C!  ( c a -- )
        POP_AX
        MASK eax
        mov edx, eax
        POP_AX
        mov byte ptr [rbx+rdx], al
        ret
```

ARM64

```
p_cstore:
        POP_W0
        and w2, w0, #0xFFFF
        POP_W0
        add x11, x19, w2, uxtw
        strb w0, [x11]
        ret
```

RISC-V (RV64IM)

```
p_cstore:
        POP_A0
        and a1, a0, s4
        POP_A0
        add t3, s0, a1
        sb a0, 0(t3)
        ret
```

`MOVE` (from to n --)

Copy n bytes, overlapping safely. Faithful to JavaScript's copyWithin: n <= 0 copies nothing, and a copy is clamped at the top of memory. Headers are made with it (the name is copied from PAD), and CLONE and CREATE-copies use it. On the 6502 it is a byte loop in either direction.

Native #13 · called 333 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	93	48	54	52
6502 cycles per call	292			

65C02

```
p_move:                         ; MOVE ( from to n ): JS copyWithin, clamped to the 64K
        POPTO T1                ; n
        POPTO T2                ; to
        POPTO T3                ; from
        LDA T1+1
        BMI mv_none             ; n <= 0: nothing
        ORA T1
        BEQ mv_none
        CLC                     ; the source runs past $FFFF ( a negative from, a non-negative end ): nothing
        LDA T3
        ADC T1
        LDA T3+1
        ADC T1+1
        BCC mv_ok
mv_none:
        RTS
mv_ok:
        CLC                     ; the target would run past $FFFF: only what fits
        LDA T2
        ADC T1
        LDA T2+1
        ADC T1+1
        BCC mv_go
        SEC
        LDA #0
        SBC T2
        STA T1
        LDA #0
        SBC T2+1
        STA T1+1
mv_go:
        LDA T3
        STA P1
        LDA T3+1
        STA P1+1
        LDA T2
        STA P2
        LDA T2+1
        STA P2+1
        LDA T2+1                ; to above from: copy backwards
        CMP T3+1
        BCC mv_fwd
        BNE mv_back
        LDA T2
        CMP T3
        BCC mv_fwd
        BEQ mv_fwd
mv_back:
        CLC
        LDA P1
```

```
        ADC T1
        STA P1
        LDA P1+1
        ADC T1+1
        STA P1+1
        CLC
        LDA P2
        ADC T1
        STA P2
        LDA P2+1
        ADC T1+1
        STA P2+1
mb1:
        LDA P1
        BNE mb2
        DEC P1+1
mb2:
        DEC P1
        LDA P2
        BNE mb3
        DEC P2+1
mb3:
        DEC P2
        LDA (P1)
        STA (P2)
        LDA T1
        BNE mb4
        DEC T1+1
mb4:
        DEC T1
        LDA T1
        ORA T1+1
        BNE mb1
mv_done:
        RTS
mv_fwd:
        LDA (P1)
        STA (P2)
        INC P1
        BNE mf1
        INC P1+1
mf1:
        INC P2
        BNE mf2
        INC P2+1
mf2:
        LDA T1
        BNE mf3
        DEC T1+1
mf3:
        DEC T1
        LDA T1
        ORA T1+1
        BNE mv_fwd
        RTS
```

x86-64

```
p_move:                                 # MOVE  ( from to n -- ): JS copyWithin, clamped to the 64K
        POP_AX
        mov r8d, eax                    # n ( signed )
        POP_AX
        mov r9d, eax                    # to
        POP_AX
        mov r10d, eax                   # from
        mov eax, r10d                   # start = from < 0 ? from + 64K : from
        test eax, eax
        jns 1f
        add eax, 0x10000
1:      lea edx, [r10+r8]               # end = from + n, the same way; clamped to 64K
        test edx, edx
        jns 2f
        add edx, 0x10000
2:      cmp edx, 0x10000
        jbe 3f
        mov edx, 0x10000
3:      mov ecx, r9d                    # to, the same way
        test ecx, ecx
        jns 4f
        add ecx, 0x10000
4:      sub edx, eax                    # count = end - start, clamped to what fits at to
        jle 9f
        mov r11d, 0x10000
        sub r11d, ecx
        cmp edx, r11d
        jle 5f
        mov edx, r11d
5:      test edx, edx
        jle 9f
        lea rsi, [rbx+rax]              # memmove: backwards when the target overlaps above the source
        lea rdi, [rbx+rcx]
        mov ecx, edx
        cmp rdi, rsi
        jbe 6f
        lea rsi, [rsi+rcx-1]
        lea rdi, [rdi+rcx-1]
        std
        rep movsb
        cld
        ret
6:      rep movsb
9:      ret
```

ARM64

```
p_move:                                 // MOVE ( from to n ): JS copyWithin, clamped to the 64K
        POP_W0
        mov w2, w0                      // n
        POP_W0
        mov w3, w0                      // to
        POP_W0
        mov w4, w0                      // from
        mov w12, #0x10000
        add w5, w4, w12                 // start = from < 0 ? from + 64K : from
        cmp w4, #0
        csel w5, w5, w4, lt
        add w6, w4, w2                  // end = from + n, the same way, clamped to 64K
        add w7, w6, w12
        cmp w6, #0
        csel w6, w7, w6, lt
        cmp w6, w12
        csel w6, w12, w6, gt
        add w7, w3, w12                 // target, the same way
        cmp w3, #0
        csel w7, w7, w3, lt
        subs w6, w6, w5                 // count
        b.le 9f
        sub w8, w12, w7
        cmp w6, w8
        csel w6, w8, w6, gt
        cmp w6, #0
        b.le 9f
        add x11, x19, w5, uxtw          // source
        add x13, x19, w7, uxtw          // target
        cmp x13, x11
        b.ls 2f
1:      subs w6, w6, #1                 // backwards: the target overlaps above the source
        ldrb w9, [x11, w6, uxtw]
        strb w9, [x13, w6, uxtw]
        b.ne 1b
        ret
2:      ldrb w9, [x11], #1              // forwards
        strb w9, [x13], #1
        subs w6, w6, #1
        b.ne 2b
9:      ret
```

RISC-V (RV64IM)

```
p_move:                                 # MOVE ( from to n ): JS copyWithin, clamped to the 64K
        POP_A0
        mv a1, a0                       # n
        POP_A0
        mv a2, a0                       # to
        POP_A0
        mv a3, a0                       # from
        li t4, 0x10000
        mv a4, a3                       # start
        bgez a4, 1f
        add a4, a4, t4
1:      add a5, a3, a1                  # end
        bgez a5, 2f
        add a5, a5, t4
2:      ble a5, t4, 3f
        mv a5, t4
3:      mv a6, a2                       # target
        bgez a6, 4f
        add a6, a6, t4
4:      sub a5, a5, a4                  # count
        blez a5, 9f
        sub t5, t4, a6
        ble a5, t5, 5f
        mv a5, t5
5:      blez a5, 9f
        add t5, s0, a4                  # source
        add t6, s0, a6                  # target
        bleu t6, t5, 7f
        add t5, t5, a5                  # backwards: the target overlaps above the source
        add t6, t6, a5
6:      addi t5, t5, -1
        addi t6, t6, -1
        lbu t3, 0(t5)
        sb t3, 0(t6)
        addi a5, a5, -1
        bnez a5, 6b
        ret
7:      lbu t3, 0(t5)                   # forwards
        sb t3, 0(t6)
        addi t5, t5, 1
        addi t6, t6, 1
        addi a5, a5, -1
        bnez a5, 7b
9:      ret
```

The 2x2 compiler

These eleven are what is unusual about ITCH, and what makes the rest of the compiler small. There is no STATE and no flag in any header. The compile stack holds a frame per open definition (outer frame, space, xt, hole) and the IF/BEGIN addresses above it; the context stack holds one mode per frame. Three of them, HERE , and PATCH, are VECTORED: when the open definition is built in another space (a target machine), they jump through that space's table instead of touching this RAM. That one test is the whole of cross-compilation in the kernel.

`HERE` (-- a)

Where the next compiled cell goes. Vectored: in another space, runs that space's HERE (the tether's THERE). The compiler asks it before every jump target. Vectoring it costs one test of CFP on the local path.

Native #38 · called 53 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	29	15	17	19
6502 cycles per call	96			

65C02

```
p_here:
        VECTORED 0, ph_local
ph_local:
        LDA HERE
        STA VAL
        LDA HERE+1
        STA VAL+1
        PUSHFROM VAL
        RTS
```

x86-64

```
p_here:                                 # HERE  ( -- a ) vectored
        VECTORED 0, 1f
1:      movzx eax, word ptr [rbx+HERE_]
        PUSH_AX
        ret
```

ARM64

```
p_here:
        VECTORED 0, 1f
1:      ldrh w0, [x19, #HERE_]
        PUSH_W0
        ret
```

RISC-V (RV64IM)

```
p_here:
        VECTORED 0, 1f
1:      lhu a0, HERE_(s0)
        PUSH_A0
        ret
```

`,` (x --)

Compile a cell at HERE. Vectored like HERE. Traps with DICTIONARY FULL at $8000. The only word that writes the dictionary's cells: vectoring it moves the whole compiler to the target.

Native #11 · called 174 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	53	21	23	26
6502 cycles per call	163			

65C02

```
p_comma:
        VECTORED 2, pc_local
pc_local:
        POPTO VAL
        JMP comma_val
```

x86-64

```
p_comma:                                # ,  ( x -- ) vectored
        VECTORED 2, 1f
1:      POP_AX
        jmp comma_ax
```

ARM64

```
p_comma:
        VECTORED 2, 1f
1:      POP_W0
        b comma_w0
```

RISC-V (RV64IM)

```
p_comma:
        VECTORED 2, 1f
1:      POP_A0
        j comma_a0
```

`PATCH` (x a --)

Fill a hole left in a definition (THEN fills IF's). Vectored like HERE. In a traditional Forth THEN does HERE SWAP ! . Making it a word of its own lets THEN work unchanged when the hole is in another machine's RAM.

Native #39 · called 3 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	59	25	29	32
6502 cycles per call	182			

65C02

```
p_patch:
        VECTORED 4, pp_local
pp_local:
        POPTO P1
        POPTO VAL
        LDY #0
        LDA VAL
        STA (P1),Y
        INY
        LDA VAL+1
        STA (P1),Y
        RTS
```

x86-64

```
p_patch:                                # PATCH  ( x a -- ) vectored
        VECTORED 4, 1f
1:      POP_AX
        MASK eax
        mov edx, eax
        POP_AX
        mov word ptr [rbx+rdx], ax
        ret
```

ARM64

```
p_patch:
        VECTORED 4, 1f
1:      POP_W0
        and w2, w0, #0xFFFF
        POP_W0
        strh w0, [x19, w2, uxtw]
        ret
```

RISC-V (RV64IM)

```
p_patch:
        VECTORED 4, 1f
1:      POP_A0
        and a1, a0, s4
        POP_A0
        add t3, s0, a1
        sh a0, 0(t3)
        ret
```

`>CS` (space xt hole --)

Open a definition: a 4-cell frame on the compile stack (outer frame end, space, xt, hole), CFP := its end, and COMPILING on the context stack. Refuses a nested word built in a different space (trap 8). BOOT's : and the tether's T-OPEN both end here: one primitive opens every kind of definition.

Native #30 · called 20 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	134	68	79	81
6502 cycles per call	430			

65C02

```
p_tocs:
        POPTO T6                ; hole
        POPTO T5                ; xt
        POPTO T4                ; space
        JMP open_frame
```

x86-64

```
p_tocs:                                 # >CSB  ( space xt hole -- ): open a definition
        POP_AX
        mov r8d, eax
        POP_AX
        mov r9d, eax
        POP_AX
        mov r10d, eax
        jmp open_frame
```

ARM64

```
p_tocs:
        POP_W0
        mov w7, w0                      // hole
        POP_W0
        mov w6, w0                      // xt
        POP_W0
        mov w5, w0                      // space
        b open_frame
```

RISC-V (RV64IM)

```
p_tocs:
        POP_A0
        mv a7, a0                       # hole
        POP_A0
        mv a6, a0                       # xt
        POP_A0
        mv a5, a0                       # space
        j open_frame
```

`(CLOSE)` (--)

Close the open definition's frame on both stacks, with no EXIT. For a local word it also fills the enclosing word's hole, puts the word back at the head of its wordlist and makes it LATEST; a nameless word leaves its xt. Another space's ; lays its own EXIT and then calls this; the kernel's ; lays EXIT and does the same thing.

Native #40 · called 2 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	1008	409	358	360
6502 cycles per call	3042			

65C02

```
p_close:
        JMP close_frame
```

x86-64

```
p_close:                            # (CLOSE)  close the frame, no EXIT ( another space's ; )
        jmp close_frame
```

ARM64

```
p_close:
        b close_frame
```

RISC-V (RV64IM)

```
p_close:
        j close_frame
```

`OPEN?` (-- f)

Is any definition open? (CFP is not 0) BOOT's (OPEN) asks it to decide whether the new word is nested and needs a BRANCH around it.

Native #31 · called 20 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	17	10	11	11
6502 cycles per call	56			

65C02

```
p_openq:
        LDA CFP
        ORA CFP+1
        BEQ oq1
        LDA #$FF
oq1:
        STA VAL
        STA VAL+1
        PUSHFROM VAL
        RTS
```

x86-64

```
p_openq:                                # OPEN?  ( -- f )
        movzx eax, word ptr [rbx+CFP_]
        neg eax
        sbb eax, eax
        PUSH_AX
        ret
```

ARM64

```
p_openq:
        ldrh w0, [x19, #CFP_]
        cmp w0, #0
        csetm w0, ne
        PUSH_W0
        ret
```

RISC-V (RV64IM)

```
p_openq:
        lhu a0, CFP_(s0)
        snez a0, a0
        neg a0, a0
        PUSH_A0
        ret
```

`COMPILING?` (-- f)

Is the top context frame a compiling one? What STATE @ is elsewhere: one indexed load of the context stack. S" and ." ask it.

Native #32 · called 3 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	24	13	13	13
6502 cycles per call	85			

65C02

```
p_compilingq:
        JSR mode_
        LDA VAL
        ORA VAL+1
        BEQ cq1
        LDA #$FF
cq1:
        STA VAL
        STA VAL+1
        PUSHFROM VAL
        RTS
```

x86-64

```
p_compilingq:                           # COMPILING?  ( -- f )
        call mode_
        neg eax
        sbb eax, eax
        PUSH_AX
        ret
```

ARM64

```
p_compilingq:
        ldrh w9, [x19, #CXP_]
        add x10, x19, w9, uxtw
        ldrh w0, [x10, #CXB-2]
        cmp w0, #0
        csetm w0, ne
        PUSH_W0
        ret
```

RISC-V (RV64IM)

```
p_compilingq:
        lhu t0, CXP_(s0)
        add t2, s0, t0
        lhu a0, CXB-2(t2)
        snez a0, a0
        neg a0, a0
        PUSH_A0
        ret
```

`[` (--)

Push an INTERPRETING frame on the context stack. Interpreting inside a definition: the word being built waits underneath, untouched.

Native #33 · called 1 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	17	9	10	10
6502 cycles per call	53			

65C02

```
p_lbrack:
        STZ VAL
        STZ VAL+1
        JMP cxpush_val
```

x86-64

```
p_lbrack:                               # [  interpret here
        xor eax, eax
        jmp cxpush_ax
```

ARM64

```
p_lbrack:
        mov w0, #0
        b cxpush_w0
```

RISC-V (RV64IM)

```
p_lbrack:
        li a0, 0
        j cxpush_a0
```

`]` (--)

Pop that frame. Traps with] without [when there is none. Because [and] are a push and a pop, they nest, and] cannot end a definition by mistake.

Native #34 · called 1 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	29	15	13	13
6502 cycles per call	88			

65C02

```
p_rbrack:
        LDA CXP+1
        BNE rb1
        LDA CXP
        CMP #3
        BCS rb1
        JMP trap_no_bracket
rb1:
        JSR mode_
        LDA VAL
        ORA VAL+1
        BEQ rb2
        JMP trap_no_bracket
rb2:
        JMP cxpop
```

x86-64

```
p_rbrack:                               # ]  back to compiling
        cmp word ptr [rbx+CXP_], 2
        jbe trap_no_bracket
        call mode_
        test eax, eax
        jnz trap_no_bracket
        jmp cxpop
```

ARM64

```
p_rbrack:
        ldrh w9, [x19, #CXP_]
        cmp w9, #2
        b.ls trap_no_bracket
        add x10, x19, w9, uxtw
        ldrh w0, [x10, #CXB-2]
        cbnz w0, trap_no_bracket
        b cxpop
```

RISC-V (RV64IM)

```
p_rbrack:
        lhu t0, CXP_(s0)
        li t1, 2
        bgtu t0, t1, 1f
        j trap_no_bracket
1:      add t2, s0, t0
        lhu a0, CXB-2(t2)
        JNZ a0, trap_no_bracket
        j cxpop
```

`>C` (x --) (C: -- x)

Push onto the compile stack (IF and BEGIN leave their addresses here). Control-flow bookkeeping never touches the data stack, so a word may compute while it compiles.

Native #36 · called 7 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	26	13	15	15
6502 cycles per call	81			

65C02

```
p_toc:
        POPTO VAL
        JMP cpush_val
```

x86-64

```
p_toc:                                  # >C  ( x -- ) ( C: -- x )
        POP_AX
        jmp cpush_ax
```

ARM64

```
p_toc:
        POP_W0
        b cpush_w0
```

RISC-V (RV64IM)

```
p_toc:
        POP_A0
        j cpush_a0
```

`C>` (-- x) (C: x --)

Pop the compile stack, never into the open frame (trap CONTROL STRUCTURE MISMATCH). This bound check is the whole of ITCH's structure checking: an ELSE without IF or a stray THEN fails here, and ; refuses to close a frame with anything left above it. BOOT's IF THEN ELSE are one line of Forth each, with no error code.

Native #37 · called 6 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	32	16	20	21
6502 cycles per call	101			

65C02

```
p_cfrom:
        JSR cpop_val
        PUSHFROM VAL
        RTS
```

x86-64

```
p_cfrom:                                # C>  ( -- x ) ( C: x -- )
        call cpop_ax
        PUSH_AX
        ret
```

ARM64

```
p_cfrom:
        ENTER
        bl cpop_w0
        PUSH_W0
        LEAVE
```

RISC-V (RV64IM)

```
p_cfrom:
        ENTER
        jal ra, cpop_a0
        PUSH_A0
        LEAVE
```

The seed: reading BOOT before Forth exists

ITCH boots from source text. Before BOOT has defined WORD, FIND or : in Forth, the kernel must read BOOT's first lines, so it carries a seed of the text interpreter. After boot, WORD and FIND are deferred words re-aimed at BOOT's Forth versions and BOOT's : replaces the seed one; ' ['] and ; stay as they are. These six are the kernel's largest routines, because they walk text and wordlists.

`(WORD)` (c -- c-addr)

Parse the next name (delimiter c) from the source into PAD as [len][chars], upper-cased, at most 127. The seed for WORD: ' (WORD) IS WORD puts the fast version back.

Native #16 · called 2 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	245	103	107	106
6502 cycles per call	799			

65C02

```
p_pword:
        POPTO T1
        JSR word_
        PUSHFROM VAL
        RTS
```

x86-64

```
p_pword:                                # (WORD)  ( c -- c-addr )
        POP_AX
        mov edi, eax
        call word_
        PUSH_AX
        ret
```

ARM64

```
p_pword:
        ENTER
        POP_W0
        bl word_
        PUSH_W0
        LEAVE
```

RISC-V (RV64IM)

```
p_pword:
        ENTER
        POP_A0
        jal ra, word_
        PUSH_A0
        LEAVE
```

`(FIND)` (c-addr -- xt 1 | xt -1 | c-addr 0)

Find a name: 1 for a word in COMMON or COMPILER (it acts while compiling), -1 for FORTH, 0 if unknown. The seed for FIND. Its cost is the walk along the wordlists: about 2,500 instructions per call here, because a name near the end of FORTH is compared with hundreds of headers (length first, bytes only when the length matches).

Native #17 · called 1 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	4668	2477	2540	2546
6502 cycles per call	14198			

65C02

```
p_pfind:
        POPTO T3
        LDA T3
        STA T6
        LDA T3+1
        STA T6+1
        STZ P2+1
        LDA #COMMON
        STA P2
        JSR find_in
        LDA VAL
        ORA VAL+1
        BNE pf_imm
        LDA #COMPILER
        STA P2
        JSR find_in
        LDA VAL
        ORA VAL+1
        BNE pf_imm
        LDA #FORTH
        STA P2
        JSR find_in
        LDA VAL
        ORA VAL+1
        BNE pf_forth
        PUSHFROM T6
        STZ VAL
        STZ VAL+1
        PUSHFROM VAL
        RTS
pf_imm:
        PUSHFROM VAL
        LDA #1
        STA VAL
        STZ VAL+1
        PUSHFROM VAL
        RTS
pf_forth:
        PUSHFROM VAL
        LDA #$FF
        STA VAL
        STA VAL+1
        PUSHFROM VAL
        RTS
```

x86-64

```
p_pfind:                                # (FIND)  ( c-addr -- xt 1 | xt -1 | c-addr 0 )
        POP_AX
        MASK eax
        mov edi, eax
        mov r11d, eax                   # keep c
        mov esi, COMMON_
        call find_in
        mov r10d, 1
        test eax, eax
        jnz 1f
        mov esi, COMPILER_
        call find_in
        test eax, eax
        jnz 1f
        mov esi, FORTH_
        call find_in
        mov r10d, -1
        test eax, eax
        jnz 1f
        mov eax, r11d                   # not found: c 0
        PUSH_AX
        xor eax, eax
        PUSH_AX
        ret
1:      PUSH_AX
        mov eax, r10d
        PUSH_AX
        ret
```

ARM64

```
p_pfind:
        ENTER
        POP_W0
        and w2, w0, #0xFFFF
        mov w7, w2                      // keep c
        mov w1, #COMMON_
        bl find_in
        mov w8, #1
        cbnz w0, 1f
        mov w1, #COMPILER_
        bl find_in
        cbnz w0, 1f
        mov w1, #FORTH_
        bl find_in
        mov w8, #-1
        cbnz w0, 1f
        mov w0, w7
        PUSH_W0
        mov w0, #0
        PUSH_W0
        LEAVE
1:      PUSH_W0
        mov w0, w8
        PUSH_W0
        LEAVE
```

RISC-V (RV64IM)

```
p_pfind:
        ENTER
        POP_A0
        and a2, a0, s4
        mv a6, a2                       # keep c
        li a1, COMMON_
        jal ra, find_in
        li a7, 1
        bnez a0, 1f
        li a1, COMPILER_
        jal ra, find_in
        bnez a0, 1f
        li a1, FORTH_
        jal ra, find_in
        li a7, -1
        bnez a0, 1f
        mv a0, a6
        PUSH_A0
        li a0, 0
        PUSH_A0
        LEAVE
1:      PUSH_A0
        mv a0, a7
        PUSH_A0
        LEAVE
```

`'` ("name" -- xt)

The xt of the next name, or the trap ' word not found. Used while reading BOOT and ever after.

Native #26 · called 6 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	1638	853	880	886
6502 cycles per call	5038			

65C02

```
p_tick:
        JSR tick_
        PUSHFROM VAL
        RTS
```

x86-64

```
p_tick:                                 # '  ( "name" -- xt )
        call tick_
        PUSH_AX
        ret
```

ARM64

```
p_tick:
        ENTER
        bl tick_
        PUSH_W0
        LEAVE
```

RISC-V (RV64IM)

```
p_tick:
        ENTER
        jal ra, tick_
        PUSH_A0
        LEAVE
```

`[']` ("name" --)

While compiling: lay LIT and the next name's xt. Lives in COMPILER, so it acts while compiling.

Native #27 · called 2 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	1622	838	866	874
6502 cycles per call	5000			

65C02

```
p_btick:
        LDA KLIT
        STA VAL
        LDA KLIT+1
        STA VAL+1
        JSR comma_val
        JSR tick_
        JMP comma_val
```

x86-64

```
p_btick:                            # [']  ( "name" -- ): LIT xt into the definition
        movzx eax, word ptr [rbx+K_LIT]
        call comma_ax
        call tick_
        jmp comma_ax
```

ARM64

```
p_btick:
        ENTER
        ldrh w0, [x22, #18]             // LIT_XT
        bl comma_w0
        bl tick_
        bl comma_w0
        LEAVE
```

RISC-V (RV64IM)

```
p_btick:
        ENTER
        lhu a0, 18(s3)                  # LIT_XT
        jal ra, comma_a0
        jal ra, tick_
        jal ra, comma_a0
        LEAVE
```

`:` ("name" --)

The seed colon: a COLON header for the next name, then its frames on both stacks. Reads BOOT until BOOT's own : (in COMMON, so it also works inside a definition) takes over.

Native #28 · called 1 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	503	270	290	303
6502 cycles per call	1654			

65C02

```
p_colon:
        LDA #32
        STA T1
        STZ T1+1
        JSR word_
        LDA VAL
        STA T3
        LDA VAL+1
        STA T3+1
        LDA #1
        STA F
        LDA KSEED
        STA T1
        LDA KSEED+1
        STA T1+1
        JSR header
        LDA VAL
        STA T5
        LDA VAL+1
        STA T5+1
        STZ T4
        STZ T4+1
        STZ T6
        STZ T6+1
        JMP open_frame
```

x86-64

```
p_colon:                            # :  the seed colon: a COLON header, frames on both stacks
        mov edi, 32
        call word_
        mov edi, eax
        mov esi, 1
        movzx edx, word ptr [rbx+K_SEED]
        call header
        mov r9d, eax                # xt
        xor r10d, r10d              # space 0
        xor r8d, r8d                # no hole
        jmp open_frame
```

ARM64

```
p_colon:
        ENTER
        mov w0, #32
        bl word_
        mov w1, #1
        ldrh w2, [x22, #22]         // SEED_COLON
        bl header
        mov w6, w0
        mov w5, #0
        mov w7, #0
        bl open_frame
        LEAVE
```

RISC-V (RV64IM)

```
p_colon:
        ENTER
        li a0, 32
        jal ra, word_
        li a1, 1
        lhu a2, 22(s3)              # SEED_COLON
        jal ra, header
        mv a6, a0
        li a5, 0
        li a7, 0
        jal ra, open_frame
        LEAVE
```

`;` (--)

Lay EXIT and close the definition. Traps when not compiling, when an IF or BEGIN is still open, or when the word is built in another space (that space has its own ;). Lives in COMPILER. The three traps are the compile stack's frame doing the

checking.

Native #29 · called 18 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	708	289	262	265
6502 cycles per call	2159			

65C02

```
p_semi:
        JSR mode_
        LDA VAL
        ORA VAL+1
        BNE sm1
        JMP trap_semi_outside
sm1:
        LDA CFP
        ORA CFP+1
        BEQ sm2
        LDA CSP
        CMP CFP
        BNE sm_open
        LDA CSP+1
        CMP CFP+1
        BEQ sm2
sm_open:
        JMP trap_semi_open
sm2:
        JSR frame_space
        LDA VAL
        ORA VAL+1
        BEQ sm3
        JMP trap_semi_space
sm3:
        LDA KEXIT
        STA VAL
        LDA KEXIT+1
        STA VAL+1
        JSR comma_val
        JMP close_frame
```

x86-64

```
p_semi:                                 # ;  lay EXIT, close the frame
        call mode_
        test eax, eax
        jz trap_semi_outside
        movzx eax, word ptr [rbx+CFP_]
        test eax, eax
        jz 1f
        cmp ax, word ptr [rbx+CSP_]
        jne trap_semi_open
1:      call frame_space
        test eax, eax
        jnz trap_semi_space
        movzx eax, word ptr [rbx+K_EXIT]
        call comma_ax
        jmp close_frame
```

ARM64

```
p_semi:
        ENTER
        bl mode_
        cbz w0, trap_semi_outside
        ldrh w0, [x19, #CFP_]
        cbz w0, 1f
        ldrh w9, [x19, #CSP_]
        cmp w9, w0
        b.ne trap_semi_open
1:      bl frame_space
        cbnz w0, trap_semi_space
        ldrh w0, [x22, #16]             // EXIT_XT
        bl comma_w0
        bl close_frame
        LEAVE
```

RISC-V (RV64IM)

```
p_semi:
        ENTER
        jal ra, mode_
        JZ a0, trap_semi_outside
        lhu a0, CFP_(s0)
        beqz a0, 1f
        lhu t0, CSP_(s0)
        beq t0, a0, 1f
        j trap_semi_open
1:      jal ra, frame_space
        JNZ a0, trap_semi_space
        lhu a0, 16(s3)                  # EXIT_XT
        jal ra, comma_a0
        jal ra, close_frame
        LEAVE
```

The machine

One primitive for the CPU itself.

`WAIT` (--)

Sleep until the next frame interrupt: set the WAIT latch and return to the host. The 65C02's WAI instruction. EMIT waits in it when the terminal's ring is full, KEY when no key has come, SEND and RECV on the UART: every device wait in the system is this one word, which is why the kernel has no I/O primitives.

Native #15 · called 1 times in the proof run

	6502	x86-64	ARM64	RISC-V
instructions per call	3	2	3	3
6502 cycles per call	12			

65C02

```
p_wait:                         ; WAIT: the 65C02 would execute WAI here
        LDA #1
        STA ASLEEP
        RTS
```

x86-64

```
p_wait:                                 # WAIT: sleep until the next frame ( the 65C02's WAI )
        mov byte ptr [rbx+K_ASLEEP], 1
        ret
```

ARM64

```
p_wait:
        mov w9, #1
        strb w9, [x22]
        ret
```

RISC-V (RV64IM)

```
p_wait:
        li t0, 1
        sb t0, 0(s3)
        ret
```

7. Shared routines

The larger primitives call these. They are where the seed's text handling and the 2x2's frames live.

65C02

```
udiv16:                         ; T2 / T1, unsigned -> VAL, remainder T3
        STZ T3
        STZ T3+1
        LDX #16
ud1:
        ASL T2
        ROL T2+1
        ROL T3
        ROL T3+1
        SEC
        LDA T3
        SBC T1
        TAY
        LDA T3+1
        SBC T1+1
        BCC ud2
        STA T3+1
        STY T3
        INC T2
ud2:
        DEX
        BNE ud1
        LDA T2
        STA VAL
        LDA T2+1
        STA VAL+1
        RTS

word_:                          ; T1 = delimiter -> VAL = PAD, the name in it upper-cased
        LDA IN
        STA T2
        LDA IN+1
        STA T2+1
        STZ N
ws1:
        JSR w_more              ; skip delimiters
        BCC wt1
        JSR w_char
        LDX T1+1
        BNE wt1
        CMP T1
        BNE wt1
        INC T2
        BNE ws1
        INC T2+1
        BRA ws1
wt1:
        JSR w_more              ; the name
        BCC we1
        JSR w_char
        LDX T1+1
        BNE wt2
        CMP T1
        BEQ we1
wt2:
        INC T2
        BNE wt3
        INC T2+1
wt3:
        LDX N
```

```
        CPX #127
        BCS wt1
        CMP #97                 ; a-z to A-Z
        BCC wt4
        CMP #123
        BCS wt4
        SEC
        SBC #32
wt4:
        STA PAD+1,X
        INC N
        BRA wt1
we1:
        JSR w_more              ; step past the delimiter
        BCC we2
        INC T2
        BNE we2
        INC T2+1
we2:
        LDA T2
        STA IN
        LDA T2+1
        STA IN+1
        LDA N
        STA PAD
        LDA #<PAD
        STA VAL
        LDA #>PAD
        STA VAL+1
        RTS

w_more:                         ; C set if i < #TIB
        LDA T2+1
        CMP NTIB+1
        BCC wm_yes
        BNE wm_no
        LDA T2
        CMP NTIB
        BCC wm_yes
wm_no:
        CLC
        RTS
wm_yes:
        SEC
        RTS

w_char:                         ; A := the byte at SRC + i
        CLC
        LDA SRC
        ADC T2
        STA P1
        LDA SRC+1
        ADC T2+1
        STA P1+1
        LDA (P1)
        RTS

find_in:                        ; P2 = a wordlist head's address, T3 = counted string -> VAL = xt or 0
        LDA (T3)
        STA N
        CLC
        LDA T3
        ADC #1
        STA T4
        LDA T3+1
        ADC #0
        STA T4+1
        LDY #0
        LDA (P2),Y
        STA P1
        INY
        LDA (P2),Y
        STA P1+1
fi_loop:
        LDA P1
        ORA P1+1
        BEQ fi_none
        LDY #2
        LDA (P1),Y
        AND #$7F
        CMP N
        BNE fi_next
        CLC
        LDA P1
        ADC #3
```

```
        STA P3
        LDA P1+1
        ADC #0
        STA P3+1
        LDY #0
fi_cmp:
        CPY N
        BEQ fi_found
        LDA (P3),Y
        CMP (T4),Y
        BNE fi_next
        INY
        BRA fi_cmp
fi_next:
        LDY #0
        LDA (P1),Y
        TAX
        INY
        LDA (P1),Y
        STA P1+1
        STX P1
        BRA fi_loop
fi_found:
        LDA P1
        STA VAL
        LDA P1+1
        STA VAL+1
        RTS
fi_none:
        STZ VAL
        STZ VAL+1
        RTS

find_all:                       ; T3 = counted string: COMMON, COMPILER, FORTH
        STZ P2+1
        LDA #COMMON
        STA P2
        JSR find_in
        LDA VAL
        ORA VAL+1
        BNE fa_done
        LDA #COMPILER
        STA P2
        JSR find_in
        LDA VAL
        ORA VAL+1
        BNE fa_done
        LDA #FORTH
        STA P2
        JSR find_in
fa_done:
        RTS

tick_:                          ; the next name's xt, or trap
        LDA #32
        STA T1
        STZ T1+1
        JSR word_
        LDA VAL
        STA T3
        LDA VAL+1
        STA T3+1
        JSR find_all
        LDA VAL
        ORA VAL+1
        BNE tk1
        JMP trap_not_found
tk1:
        RTS

cpush_val:
        LDA CSP+1
        BEQ cp1
        JMP trap_cs_over
cp1:
        LDX CSP
        LDA VAL
        STA CSB,X
        LDA VAL+1
        STA CSB+1,X
        INX
        INX
        STX CSP
        BNE cp2
        INC CSP+1
```

```
cp2:
        RTS

cpop_val:                       ; never into the open frame
        LDA CSP
        SEC
        SBC #2
        TAX
        LDA CSP+1
        SBC #0
        BMI cpb
        CMP CFP+1
        BCC cpb
        BNE cpo
        CPX CFP
        BCC cpb
cpo:
        STA CSP+1
        STX CSP
        LDA CSB,X
        STA VAL
        LDA CSB+1,X
        STA VAL+1
        RTS
cpb:
        JMP trap_mismatch

cxpush_val:
        LDA CXP+1
        BNE cxo
        LDA CXP
        CMP #64
        BCC cx1
cxo:
        JMP trap_cx_over
cx1:
        TAX
        LDA VAL
        STA CXB,X
        LDA VAL+1
        STA CXB+1,X
        INX
        INX
        STX CXP
        RTS

cxpop:
        LDA CXP
        SEC
        SBC #2
        TAX
        LDA CXP+1
        SBC #0
        BMI cxu
        BNE cxk
        CPX #2
        BCC cxu
cxk:
        STA CXP+1
        STX CXP
        RTS
cxu:
        JMP trap_cx_under

mode_:                          ; VAL := the top context frame's mode
        LDX CXP
        LDA CXB-2,X
        STA VAL
        LDA CXB-1,X
        STA VAL+1
        RTS

frame_space:                    ; VAL := the open frame's space, or 0
        LDA CFP
        ORA CFP+1
        BNE fs1
        STZ VAL
        STZ VAL+1
        RTS
fs1:
        CLC
        LDA CFP
        ADC #<(CSB-6)
        STA P3
        LDA CFP+1
```

```
        ADC #>(CSB-6)
        STA P3+1
        LDY #0
        LDA (P3),Y
        STA VAL
        INY
        LDA (P3),Y
        STA VAL+1
        RTS

open_frame:                     ; T4 = space, T5 = xt, T6 = hole
        LDA CFP
        ORA CFP+1
        BEQ of1
        JSR frame_space
        LDA VAL
        CMP T4
        BNE of_n
        LDA VAL+1
        CMP T4+1
        BEQ of1
of_n:
        JMP trap_nested
of1:
        LDA CFP
        STA VAL
        LDA CFP+1
        STA VAL+1
        JSR cpush_val
        LDA T4
        STA VAL
        LDA T4+1
        STA VAL+1
        JSR cpush_val
        LDA T5
        STA VAL
        LDA T5+1
        STA VAL+1
        JSR cpush_val
        LDA T6
        STA VAL
        LDA T6+1
        STA VAL+1
        JSR cpush_val
        LDA CSP                 ; CFP marks the frame's end, as a frame pointer
        STA CFP
        LDA CSP+1
        STA CFP+1
        LDA #1
        STA VAL
        STZ VAL+1
        JMP cxpush_val

promote:                        ; T5 = xt: LATEST, and the head of its wordlist
        LDA T5
        STA LATEST
        LDA T5+1
        STA LATEST+1
        STZ P2+1
        LDA #COMMON
        STA P2
pr_list:
        LDY #0
        LDA (P2),Y
        STA P1
        INY
        LDA (P2),Y
        STA P1+1
        LDA P1
        CMP T5
        BNE pr_walk
        LDA P1+1
        CMP T5+1
        BEQ pr_ret
pr_walk:
        LDA P1
        ORA P1+1
        BEQ pr_next
        LDY #0
        LDA (P1),Y
        STA VAL
        INY
        LDA (P1),Y
        STA VAL+1
        LDA VAL
```

```
        CMP T5
        BNE pr_adv
        LDA VAL+1
        CMP T5+1
        BEQ pr_found
pr_adv:
        LDA VAL
        STA P1
        LDA VAL+1
        STA P1+1
        BRA pr_walk
pr_found:
        LDY #0                  ; b's LINK := xt's LINK; xt's LINK := the head; head := xt
        LDA (T5),Y
        STA (P1),Y
        INY
        LDA (T5),Y
        STA (P1),Y
        LDY #0
        LDA (P2),Y
        STA (T5),Y
        INY
        LDA (P2),Y
        STA (T5),Y
        LDY #0
        LDA T5
        STA (P2),Y
        INY
        LDA T5+1
        STA (P2),Y
pr_ret:
        RTS
pr_next:
        INC P2
        INC P2
        LDA P2
        CMP #FORTH+1
        BCC pr_list
        RTS

close_frame:
        LDA CFP
        ORA CFP+1
        BNE cf1
        JMP trap_semi_outside
cf1:
        LDA CSP
        CMP CFP
        BNE cf_open
        LDA CSP+1
        CMP CFP+1
        BEQ cf2
cf_open:
        JMP trap_semi_open
cf2:
        CLC
        LDA CFP
        ADC #<(CSB-8)
        STA P1
        LDA CFP+1
        ADC #>(CSB-8)
        STA P1+1
        LDY #2
        LDA (P1),Y
        STA T4                  ; space
        INY
        LDA (P1),Y
        STA T4+1
        INY
        LDA (P1),Y
        STA T5                  ; xt
        INY
        LDA (P1),Y
        STA T5+1
        INY
        LDA (P1),Y
        STA T6                  ; hole
        INY
        LDA (P1),Y
        STA T6+1
        LDY #0
        LDA (P1),Y
        TAX
        INY
        LDA (P1),Y
```

```
        STA CFP+1               ; CFP := the outer frame's end
        STX CFP
        SEC                     ; CSP := f - 8
        LDA CSP
        SBC #8
        STA CSP
        LDA CSP+1
        SBC #0
        STA CSP+1
        JSR cxpop
        LDA T4
        ORA T4+1
        BNE cf_done             ; another space: its own ; did the rest
        LDA T6
        ORA T6+1
        BEQ cf3
        LDY #0                  ; the enclosing word's BRANCH lands just past us
        LDA HERE
        STA (T6),Y
        INY
        LDA HERE+1
        STA (T6),Y
cf3:
        JSR promote
        LDY #2
        LDA (T5),Y
        AND #$7F
        BNE cf_done
        PUSHFROM T5             ; :NONAME leaves its xt
cf_done:
        RTS

comma_val:                      ; the dictionary's own ,
        CLC
        LDA HERE
        ADC #2
        STA T5
        LDA HERE+1
        ADC #0
        STA T5+1
        BCS cv_full
        CMP #$80
        BCC cv_ok
        BNE cv_full
        LDA T5
        BEQ cv_ok
cv_full:
        JMP trap_dict_full
cv_ok:
        LDY #0
        LDA VAL
        STA (HERE),Y
        INY
        LDA VAL+1
        STA (HERE),Y
        LDA T5
        STA HERE
        LDA T5+1
        STA HERE+1
        RTS

header:                         ; T3 = counted name, F = KIND, T1 = PARENT -> VAL = xt
        LDY #0
        LDA T1
        STA (HERE),Y            ; PARENT
        INY
        LDA T1+1
        STA (HERE),Y
        CLC
        LDA HERE
        ADC #2
        STA P1                  ; x
        LDA HERE+1
        ADC #0
        STA P1+1
        LDA (T3)
        STA N                   ; n
        LDA CURRENT
        STA P2
        LDA CURRENT+1
        STA P2+1
        LDY #0                  ; LINK := the CURRENT list's head
        LDA (P2),Y
        STA (P1),Y
        INY
```

```
        LDA (P2),Y
        STA (P1),Y
        CLC
        LDA P1
        ADC #2
        STA P3
        LDA P1+1
        ADC #0
        STA P3+1
        LDY #0
hd1:
        LDA (T3),Y              ; LEN NAME
        STA (P3),Y
        CPY N
        BEQ hd2
        INY
        BRA hd1
hd2:
        INY
        LDA F
        STA (P3),Y              ; KIND
        CLC
        LDA N
        ADC #4
        ADC P1
        STA HERE
        LDA P1+1
        ADC #0
        STA HERE+1
        LDY #0
        LDA P1
        STA (P2),Y
        STA LATEST
        INY
        LDA P1+1
        STA (P2),Y
        STA LATEST+1
        LDA P1
        STA VAL
        LDA P1+1
        STA VAL+1
        RTS
```

x86-64

```
word_:
        movzx r8d, word ptr [rbx+SRC_]  # src
        movzx r9d, word ptr [rbx+NTIB_] # end
        movzx r10d, word ptr [rbx+IN_]  # i
        xor r11d, r11d                  # n
1:      cmp r10d, r9d                   # skip delimiters
        jge 2f
        lea eax, [r8+r10]
        MASK eax
        movzx eax, byte ptr [rbx+rax]
        cmp eax, edi
        jne 2f
        inc r10d
        jmp 1b
2:      cmp r10d, r9d                   # take the name
        jge 4f
        lea eax, [r8+r10]
        MASK eax
        movzx eax, byte ptr [rbx+rax]
        cmp eax, edi
        je 4f
        inc r10d
        cmp r11d, 127                   # at most 127 kept
        jge 2b
        lea edx, [rax-97]               # a-z → A-Z
        cmp edx, 25
        ja 3f
        sub eax, 32
3:      mov byte ptr [rbx+r11+PAD+1], al
        inc r11d
        jmp 2b
4:      cmp r10d, r9d                   # step past the delimiter, if there is one
        jge 5f
        inc r10d
5:      mov word ptr [rbx+IN_], r10w
        mov byte ptr [rbx+PAD], r11b
        mov eax, PAD
        ret
```

```
# find_in: esi = a wordlist head's address, edi = a counted string. eax := its header, or 0

find_in:
        movzx r8d, byte ptr [rbx+rdi]   # n
        movzx eax, word ptr [rbx+rsi]   # x: the newest header
1:      test eax, eax
        jz 9f
        movzx edx, byte ptr [rbx+rax+2] # same length?
        and edx, 127
        cmp edx, r8d
        jne 3f
        xor ecx, ecx                    # same bytes?
2:      cmp ecx, r8d
        je 9f
        lea edx, [rax+rcx+3]
        MASK edx
        movzx r9d, byte ptr [rbx+rdx]
        lea edx, [rdi+rcx+1]
        MASK edx
        movzx r10d, byte ptr [rbx+rdx]
        cmp r9d, r10d
        jne 3f
        inc ecx
        jmp 2b
3:      movzx eax, word ptr [rbx+rax]   # LINK: the next older
        jmp 1b
9:      ret

find_all:                               # edi = counted string: COMMON, COMPILER, then FORTH. eax := xt or 0
        mov esi, COMMON_
        call find_in
        test eax, eax
        jnz 9f
        mov esi, COMPILER_
        call find_in
        test eax, eax
        jnz 9f
        mov esi, FORTH_
        call find_in
9:      ret

tick_:                                  # the next name's xt, or trap ' word not found
        mov edi, 32
        call word_
        mov edi, eax
        call find_all
        test eax, eax
        jz trap_not_found
        ret

cpush_ax:                               # cPush(ax)
        movzx ecx, word ptr [rbx+CSP_]
        cmp ecx, 256
        jae trap_cs_over
        mov word ptr [rbx+rcx+CSB], ax
        add ecx, 2
        mov word ptr [rbx+CSP_], cx
        ret

cpop_ax:                                # eax := cPop(): never into the open frame
        movzx ecx, word ptr [rbx+CSP_]
        sub ecx, 2
        movzx edx, word ptr [rbx+CFP_]
        cmp ecx, edx                    # signed: an empty stack gives -2
        jl trap_mismatch
        mov word ptr [rbx+CSP_], cx
        movzx eax, word ptr [rbx+rcx+CSB]
        ret

cxpush_ax:                              # cxPush(ax)
        movzx ecx, word ptr [rbx+CXP_]
        cmp ecx, 64
        jae trap_cx_over
        mov word ptr [rbx+rcx+CXB], ax
        add ecx, 2
        mov word ptr [rbx+CXP_], cx
        ret

cxpop:                                  # cxPop(): its bottom frame never goes
        movzx ecx, word ptr [rbx+CXP_]
        sub ecx, 2
        cmp ecx, 2
        jl trap_cx_under
        mov word ptr [rbx+CXP_], cx
        ret
```

```
mode_:                                  # eax := the top context frame's mode
        movzx eax, word ptr [rbx+CXP_]
        movzx eax, word ptr [rbx+rax+CXB-2]
        ret

frame_space:                            # eax := the open frame's space, or 0 with none open
        movzx eax, word ptr [rbx+CFP_]
        test eax, eax
        jz 1f
        movzx eax, word ptr [rbx+rax+CSB-6]
1:      ret
# open_frame: r10d = space ( as popped ), r9d = xt, r8d = hole

open_frame:
        cmp word ptr [rbx+CFP_], 0
        je 1f
        call frame_space
        cmp eax, r10d
        jne trap_nested                 # a nested word is built where the word around it is
1:      movzx eax, word ptr [rbx+CFP_]
        call cpush_ax
        mov eax, r10d
        call cpush_ax
        mov eax, r9d
        call cpush_ax
        mov eax, r8d
        call cpush_ax
        movzx eax, word ptr [rbx+CSP_]
        mov word ptr [rbx+CFP_], ax     # CFP marks the frame's end, as a frame pointer
        mov eax, 1
        jmp cxpush_ax                   # COMPILING on the context stack
# promote: edi = xt. It becomes LATEST, and goes back to the head of its wordlist

promote:
        mov word ptr [rbx+LATEST_], di
        mov esi, COMMON_
1:      movzx eax, word ptr [rbx+rsi]
        cmp eax, edi
        je 9f                           # already the head
2:      test eax, eax
        jz 4f
        movzx edx, word ptr [rbx+rax]
        cmp edx, edi
        je 3f
        mov eax, edx
        jmp 2b
3:      movzx edx, word ptr [rbx+rdi]   # b's LINK := xt's LINK; xt's LINK := the head; head := xt
        mov word ptr [rbx+rax], dx
        movzx edx, word ptr [rbx+rsi]
        mov word ptr [rbx+rdi], dx
        mov word ptr [rbx+rsi], di
        ret
4:      add esi, 2                      # the next wordlist
        cmp esi, FORTH_
        jbe 1b
9:      ret

close_frame:                            # close the open definition on both stacks ( no EXIT )
        movzx edx, word ptr [rbx+CFP_]  # f
        test edx, edx
        jz trap_semi_outside
        cmp dx, word ptr [rbx+CSP_]
        jne trap_semi_open
        movzx r8d, word ptr [rbx+rdx+CSB-6]      # space
        movzx edi, word ptr [rbx+rdx+CSB-4]      # xt
        movzx r9d, word ptr [rbx+rdx+CSB-2]      # hole
        movzx eax, word ptr [rbx+rdx+CSB-8]
        mov word ptr [rbx+CFP_], ax
        lea eax, [rdx-8]
        mov word ptr [rbx+CSP_], ax
        call cxpop
        test r8d, r8d
        jnz 9f                          # another space: its own ; did the rest
        test r9d, r9d
        jz 1f
        movzx eax, word ptr [rbx+HERE_] # the enclosing word's BRANCH lands just past us
        mov word ptr [rbx+r9], ax
1:      call promote
        test byte ptr [rbx+rdi+2], 127
        jnz 9f
        mov eax, edi                    # :NONAME leaves its xt
        PUSH_AX
9:      ret
```

```
comma_ax:                               # COMMA(ax): the dictionary's own , ( not vectored )
        movzx ecx, word ptr [rbx+HERE_]
        lea edx, [rcx+2]
        cmp edx, TEXT
        ja trap_dict_full
        mov word ptr [rbx+rcx], ax
        mov word ptr [rbx+HERE_], dx
        ret
# header: edi = counted name, esi = KIND, edx = PARENT. eax := the new xt

header:
        movzx ecx, word ptr [rbx+HERE_]
        mov word ptr [rbx+rcx], dx      # PARENT
        add ecx, 2
        MASK ecx                        # x
        movzx r8d, byte ptr [rbx+rdi]   # n
        movzx r9d, word ptr [rbx+CURRENT_]
        movzx eax, word ptr [rbx+r9]
        mov word ptr [rbx+rcx], ax      # LINK := the CURRENT list's head
        xor eax, eax
1:      cmp eax, r8d                    # LEN NAME, n+1 bytes
        ja 2f
        lea edx, [rdi+rax]
        MASK edx
        movzx edx, byte ptr [rbx+rdx]
        lea r10d, [rcx+rax+2]
        MASK r10d
        mov byte ptr [rbx+r10], dl
        inc eax
        jmp 1b
2:      lea r10d, [rcx+r8+3]            # KIND
        MASK r10d
        mov byte ptr [rbx+r10], sil
        lea eax, [rcx+r8+4]
        mov word ptr [rbx+HERE_], ax
        mov word ptr [rbx+r9], cx       # the list's head := x
        mov word ptr [rbx+LATEST_], cx
        mov eax, ecx
        ret
# vectored: the open frame's space table, if any, has the word; else the local version runs
```

ARM64

```
word_:                                  // w0 = delimiter ( signed ) -> w0 = PAD
        ldrh w2, [x19, #SRC_]
        ldrh w3, [x19, #NTIB_]
        ldrh w4, [x19, #IN_]
        mov w5, #0                      // n
        add x6, x19, #PAD               // PAD's address
1:      cmp w4, w3                      // skip delimiters
        b.ge 2f
        add w9, w2, w4
        and w9, w9, #0xFFFF
        ldrb w9, [x19, w9, uxtw]
        cmp w9, w0
        b.ne 2f
        add w4, w4, #1
        b 1b
2:      cmp w4, w3                      // the name
        b.ge 4f
        add w9, w2, w4
        and w9, w9, #0xFFFF
        ldrb w9, [x19, w9, uxtw]
        cmp w9, w0
        b.eq 4f
        add w4, w4, #1
        cmp w5, #127
        b.ge 2b
        sub w11, w9, #97                // a-z to A-Z
        cmp w11, #25
        b.hi 3f
        sub w9, w9, #32
3:      add w5, w5, #1
        strb w9, [x6, w5, uxtw]
        b 2b
4:      cmp w4, w3
        b.ge 5f
        add w4, w4, #1
5:      strh w4, [x19, #IN_]
        strb w5, [x6]
        mov w0, #PAD
        ret
```

```
find_in:                                // w1 = a wordlist head's address, w2 = counted string -> w0 = xt or 0
        ldrb w3, [x19, w2, uxtw]        // n
        ldrh w0, [x19, w1, uxtw]
1:      cbz w0, 9f
        add w9, w0, #2
        ldrb w9, [x19, w9, uxtw]
        and w9, w9, #127
        cmp w9, w3
        b.ne 3f
        mov w4, #0
2:      cmp w4, w3
        b.eq 9f
        add w9, w0, w4
        add w9, w9, #3
        and w9, w9, #0xFFFF
        ldrb w9, [x19, w9, uxtw]
        add w11, w2, w4
        add w11, w11, #1
        and w11, w11, #0xFFFF
        ldrb w11, [x19, w11, uxtw]
        cmp w9, w11
        b.ne 3f
        add w4, w4, #1
        b 2b
3:      ldrh w0, [x19, w0, uxtw]        // LINK
        b 1b
9:      ret

find_all:                               // w2 = counted string: COMMON, COMPILER, FORTH
        ENTER
        mov w1, #COMMON_
        bl find_in
        cbnz w0, 9f
        mov w1, #COMPILER_
        bl find_in
        cbnz w0, 9f
        mov w1, #FORTH_
        bl find_in
9:      LEAVE

tick_:                                  // the next name's xt, or trap
        ENTER
        mov w0, #32
        bl word_
        mov w2, w0
        bl find_all
        cbz w0, trap_not_found
        LEAVE

cpush_w0:
        ldrh w9, [x19, #CSP_]
        cmp w9, #256
        b.hs trap_cs_over
        add x10, x19, w9, uxtw
        strh w0, [x10, #CSB]
        add w9, w9, #2
        strh w9, [x19, #CSP_]
        ret

cpop_w0:                                // never into the open frame
        ldrh w9, [x19, #CSP_]
        sub w9, w9, #2
        ldrh w11, [x19, #CFP_]
        cmp w9, w11
        b.lt trap_mismatch
        strh w9, [x19, #CSP_]
        add x10, x19, w9, uxtw
        ldrh w0, [x10, #CSB]
        ret

cxpush_w0:
        ldrh w9, [x19, #CXP_]
        cmp w9, #64
        b.hs trap_cx_over
        add x10, x19, w9, uxtw
        strh w0, [x10, #CXB]
        add w9, w9, #2
        strh w9, [x19, #CXP_]
        ret

cxpop:
        ldrh w9, [x19, #CXP_]
        sub w9, w9, #2
        cmp w9, #2
        b.lt trap_cx_under
```

```
        strh w9, [x19, #CXP_]
        ret

mode_:                                  // w0 := the top context frame's mode
        ldrh w9, [x19, #CXP_]
        add x10, x19, w9, uxtw
        ldrh w0, [x10, #CXB-2]
        ret

frame_space:                            // w0 := the open frame's space, or 0
        ldrh w0, [x19, #CFP_]
        cbz w0, 1f
        add x10, x19, w0, uxtw
        ldrh w0, [x10, #CSB-6]
1:      ret

open_frame:                             // w5 = space ( as popped ), w6 = xt, w7 = hole
        ENTER
        ldrh w0, [x19, #CFP_]
        cbz w0, 1f
        bl frame_space
        cmp w0, w5
        b.ne trap_nested
1:      ldrh w0, [x19, #CFP_]
        bl cpush_w0
        mov w0, w5
        bl cpush_w0
        mov w0, w6
        bl cpush_w0
        mov w0, w7
        bl cpush_w0
        ldrh w0, [x19, #CSP_]
        strh w0, [x19, #CFP_]
        mov w0, #1
        bl cxpush_w0
        LEAVE

promote:                                // w6 = xt: LATEST, and the head of its wordlist
        strh w6, [x19, #LATEST_]
        mov w1, #COMMON_
1:      ldrh w0, [x19, w1, uxtw]
        cmp w0, w6
        b.eq 9f
2:      cbz w0, 4f
        ldrh w2, [x19, w0, uxtw]
        cmp w2, w6
        b.eq 3f
        mov w0, w2
        b 2b
3:      ldrh w2, [x19, w6, uxtw]
        strh w2, [x19, w0, uxtw]
        ldrh w2, [x19, w1, uxtw]
        strh w2, [x19, w6, uxtw]
        strh w6, [x19, w1, uxtw]
        ret
4:      add w1, w1, #2
        cmp w1, #FORTH_
        b.ls 1b
9:      ret

close_frame:
        ENTER
        ldrh w2, [x19, #CFP_]           // f
        cbz w2, trap_semi_outside
        ldrh w9, [x19, #CSP_]
        cmp w9, w2
        b.ne trap_semi_open
        add x10, x19, w2, uxtw
        ldrh w5, [x10, #CSB-6]          // space
        ldrh w6, [x10, #CSB-4]          // xt
        ldrh w7, [x10, #CSB-2]          // hole
        ldrh w0, [x10, #CSB-8]
        strh w0, [x19, #CFP_]
        sub w0, w2, #8
        strh w0, [x19, #CSP_]
        bl cxpop
        cbnz w5, 9f
        cbz w7, 1f
        ldrh w0, [x19, #HERE_]
        strh w0, [x19, w7, uxtw]
1:      bl promote
        add w9, w6, #2
        ldrb w9, [x19, w9, uxtw]
        tst w9, #127
        b.ne 9f
```

```
        mov w0, w6
        PUSH_W0
9:      LEAVE

comma_w0:                               // the dictionary's own ,
        ldrh w9, [x19, #HERE_]
        add w11, w9, #2
        cmp w11, #0x8000
        b.hi trap_dict_full
        strh w0, [x19, w9, uxtw]
        strh w11, [x19, #HERE_]
        ret

header:                                 // w0 = counted name, w1 = KIND, w2 = PARENT -> w0 = xt
        ldrh w3, [x19, #HERE_]
        strh w2, [x19, w3, uxtw]
        add w3, w3, #2
        and w3, w3, #0xFFFF             // x
        ldrb w4, [x19, w0, uxtw]        // n
        ldrh w5, [x19, #CURRENT_]
        ldrh w9, [x19, w5, uxtw]
        strh w9, [x19, w3, uxtw]        // LINK
        mov w6, #0
1:      cmp w6, w4                      // LEN NAME
        b.hi 2f
        add w9, w0, w6
        and w9, w9, #0xFFFF
        ldrb w9, [x19, w9, uxtw]
        add w11, w3, w6
        add w11, w11, #2
        and w11, w11, #0xFFFF
        strb w9, [x19, w11, uxtw]
        add w6, w6, #1
        b 1b
2:      add w11, w3, w4
        add w11, w11, #3
        and w11, w11, #0xFFFF
        strb w1, [x19, w11, uxtw]       // KIND
        add w11, w3, w4
        add w11, w11, #4
        strh w11, [x19, #HERE_]
        strh w3, [x19, w5, uxtw]
        strh w3, [x19, #LATEST_]
        mov w0, w3
        ret
```

RISC-V (RV64IM)

```
word_:                                  # a0 = delimiter ( signed ) -> a0 = PAD
        lhu a1, SRC_(s0)
        lhu a2, NTIB_(s0)
        lhu a3, IN_(s0)
        li a4, 0                        # n
        addi a5, s0, PAD
1:      bge a3, a2, 2f                  # skip delimiters
        add t3, a1, a3
        and t3, t3, s4
        add t3, t3, s0
        lbu t3, 0(t3)
        bne t3, a0, 2f
        addi a3, a3, 1
        j 1b
2:      bge a3, a2, 4f                  # the name
        add t3, a1, a3
        and t3, t3, s4
        add t3, t3, s0
        lbu t3, 0(t3)
        beq t3, a0, 4f
        addi a3, a3, 1
        li t4, 127
        bge a4, t4, 2b
        addi t4, t3, -97                # a-z to A-Z
        li t5, 25
        bgtu t4, t5, 3f
        addi t3, t3, -32
3:      addi a4, a4, 1
        add t4, a5, a4
        sb t3, 0(t4)
        j 2b
4:      bge a3, a2, 5f
        addi a3, a3, 1
5:      sh a3, IN_(s0)
        sb a4, 0(a5)
```

```
        li a0, PAD
        ret

find_in:                                # a1 = a wordlist head's address, a2 = counted -> a0 = xt or 0
        add t3, s0, a2
        lbu a3, 0(t3)                   # n
        add t3, s0, a1
        lhu a0, 0(t3)
1:      beqz a0, 9f
        add t3, s0, a0
        lbu t4, 2(t3)
        andi t4, t4, 127
        bne t4, a3, 3f
        li a4, 0
2:      beq a4, a3, 9f
        add t4, a0, a4
        addi t4, t4, 3
        and t4, t4, s4
        add t4, t4, s0
        lbu t4, 0(t4)
        add t5, a2, a4
        addi t5, t5, 1
        and t5, t5, s4
        add t5, t5, s0
        lbu t5, 0(t5)
        bne t4, t5, 3f
        addi a4, a4, 1
        j 2b
3:      add t3, s0, a0                  # LINK
        lhu a0, 0(t3)
        j 1b
9:      ret

find_all:
        ENTER
        li a1, COMMON_
        jal ra, find_in
        bnez a0, 9f
        li a1, COMPILER_
        jal ra, find_in
        bnez a0, 9f
        li a1, FORTH_
        jal ra, find_in
9:      LEAVE

tick_:
        ENTER
        li a0, 32
        jal ra, word_
        mv a2, a0
        jal ra, find_all
        JZ a0, trap_not_found
        LEAVE

cpush_a0:
        lhu t0, CSP_(s0)
        li t1, 256
        bltu t0, t1, 1f
        j trap_cs_over
1:      add t2, s0, t0
        sh a0, CSB(t2)
        addi t0, t0, 2
        sh t0, CSP_(s0)
        ret

cpop_a0:                                # never into the open frame
        lhu t0, CSP_(s0)
        addi t0, t0, -2
        lhu t1, CFP_(s0)
        bge t0, t1, 1f
        j trap_mismatch
1:      sh t0, CSP_(s0)
        add t2, s0, t0
        lhu a0, CSB(t2)
        ret

cxpush_a0:
        lhu t0, CXP_(s0)
        li t1, 64
        bltu t0, t1, 1f
        j trap_cx_over
1:      add t2, s0, t0
        sh a0, CXB(t2)
        addi t0, t0, 2
        sh t0, CXP_(s0)
```

```
        ret

cxpop:
        lhu t0, CXP_(s0)
        addi t0, t0, -2
        li t1, 2
        bge t0, t1, 1f
        j trap_cx_under
1:      sh t0, CXP_(s0)
        ret

mode_:
        lhu t0, CXP_(s0)
        add t2, s0, t0
        lhu a0, CXB-2(t2)
        ret

frame_space:
        lhu a0, CFP_(s0)
        beqz a0, 1f
        add t2, s0, a0
        lhu a0, CSB-6(t2)
1:      ret

open_frame:                             # a5 = space ( as popped ), a6 = xt, a7 = hole
        ENTER
        lhu a0, CFP_(s0)
        beqz a0, 1f
        jal ra, frame_space
        beq a0, a5, 1f
        j trap_nested
1:      lhu a0, CFP_(s0)
        jal ra, cpush_a0
        mv a0, a5
        jal ra, cpush_a0
        mv a0, a6
        jal ra, cpush_a0
        mv a0, a7
        jal ra, cpush_a0
        lhu a0, CSP_(s0)
        sh a0, CFP_(s0)
        li a0, 1
        jal ra, cxpush_a0
        LEAVE

promote:                                # a6 = xt
        sh a6, LATEST_(s0)
        li a1, COMMON_
1:      add t3, s0, a1
        lhu a0, 0(t3)
        beq a0, a6, 9f
2:      beqz a0, 4f
        add t3, s0, a0
        lhu a2, 0(t3)
        beq a2, a6, 3f
        mv a0, a2
        j 2b
3:      add t4, s0, a6                  # b's LINK := xt's LINK; xt's LINK := head; head := xt
        lhu a2, 0(t4)
        add t3, s0, a0
        sh a2, 0(t3)
        add t5, s0, a1
        lhu a2, 0(t5)
        sh a2, 0(t4)
        sh a6, 0(t5)
        ret
4:      addi a1, a1, 2
        li t3, FORTH_
        bleu a1, t3, 1b
9:      ret

close_frame:
        ENTER
        lhu a1, CFP_(s0)                # f
        JZ a1, trap_semi_outside
        lhu t0, CSP_(s0)
        beq t0, a1, 1f
        j trap_semi_open
1:      add t2, s0, a1
        lhu a5, CSB-6(t2)               # space
        lhu a6, CSB-4(t2)               # xt
        lhu a7, CSB-2(t2)               # hole
        lhu a0, CSB-8(t2)
        sh a0, CFP_(s0)
        addi a0, a1, -8
```

```
        sh a0, CSP_(s0)
        jal ra, cxpop
        bnez a5, 9f
        beqz a7, 2f
        lhu a0, HERE_(s0)
        add t3, s0, a7
        sh a0, 0(t3)
2:      jal ra, promote
        add t3, s0, a6
        lbu t3, 2(t3)
        andi t3, t3, 127
        bnez t3, 9f
        mv a0, a6
        PUSH_A0
9:      LEAVE

comma_a0:
        lhu t0, HERE_(s0)
        addi t1, t0, 2
        li t2, 0x8000
        bleu t1, t2, 1f
        j trap_dict_full
1:      add t3, s0, t0
        sh a0, 0(t3)
        sh t1, HERE_(s0)
        ret

header:                                 # a0 = counted name, a1 = KIND, a2 = PARENT -> a0 = xt
        lhu a3, HERE_(s0)
        add t3, s0, a3
        sh a2, 0(t3)
        addi a3, a3, 2
        and a3, a3, s4                  # x
        add t3, s0, a0
        lbu a4, 0(t3)                   # n
        lhu a5, CURRENT_(s0)
        add t3, s0, a5
        lhu t4, 0(t3)
        add t3, s0, a3
        sh t4, 0(t3)                    # LINK
        li a6, 0
1:      bgtu a6, a4, 2f                 # LEN NAME
        add t3, a0, a6
        and t3, t3, s4
        add t3, t3, s0
        lbu t4, 0(t3)
        add t3, a3, a6
        addi t3, t3, 2
        and t3, t3, s4
        add t3, t3, s0
        sb t4, 0(t3)
        addi a6, a6, 1
        j 1b
2:      add t3, a3, a4
        addi t3, t3, 3
        and t3, t3, s4
        add t3, t3, s0
        sb a1, 0(t3)                    # KIND
        add t3, a3, a4
        addi t3, t3, 4
        sh t3, HERE_(s0)
        add t3, s0, a5
        sh a3, 0(t3)
        sh a3, LATEST_(s0)
        mv a0, a3
        ret
```

8. Implementation proof

The claim: each of the four kernels above is a faithful implementation of ITCH's 41 primitives and inner interpreter. **The test:** give each one the machine the JavaScript kernel booted, run the same Forth through the real QUIT, and compare everything.

1. **The reference.** `make_ref.js` loads the JavaScript kernel and the terminal chip from ITCH-TINY.html, boots BOOT and NUMBERS, and saves the 64K image. Then it runs the 17 test texts below through `interpretSource`, exactly as the page does, and records for each one the output, the error message and **all 65,536 bytes of RAM** afterwards.
2. **The ports.** `run_proof.py` gives each kernel that image and does what the page does around it: copy the text to $8000, set TXT and TXTEND, enter QUIT through its vector, give the machine a frame whenever WAIT puts it to sleep (

draining the terminal's ring), and read MSG at the end. The tests run one after another on the same machine, so each one starts from the state the last one left.

3. **The verdict.** A test passes only if the output, the error and every byte of RAM match the JavaScript machine's.

#	Forth text	Output	What it exercises
1	`2 3 + . 7 2 - . 6 7 * . -7 2 / . 7 0 / . -32768 -1 / . 12 10 AND .`	5 5 42 -3 0 -32768 8	+ - * / AND, division by zero, -32768 / -1
2	`: SQ DUP * ; 7 SQ . 1 2 SWAP . . 5 DUP . . 9 8 DROP .`	49 1 2 5 5 9	a colon word, SWAP DUP DROP
3	`VARIABLE V 1234 V ! V @ . 65 V C! V C@ . V 1+ C@ . 42 CONSTANT K K .`	1234 65 4 42	VARIABLE, @ ! C@ C!, CONSTANT (DODOES and (UNSELF))
4	`: T 5 0 > IF 1 ELSE 2 THEN . ; T : C 3 BEGIN DUP . 1- DUP 0= UNTIL …`	1 3 2 1	IF ELSE THEN, BEGIN UNTIL (0BRANCH BRANCH >C C> PATCH)
5	`: R 5 >R 14 @ RP! R> . ; R : BR [ 2 3 + ] LITERAL . ; BR 4 ' DUP EX…`	5 5 4 4 9	>R RP! R>, [ ] LITERAL, ' EXECUTE, [']
6	`: OUT 1 . : IN 2 . ; 3 . ; OUT IN :NONAME 11 . ; EXECUTE`	1 3 2 11	a nested definition (the hole), :NONAME
7	`1 0 3 UM/MOD . . : W 32 (WORD) COUNT TYPE ; W HELLO : F 32 (WORD) (…`	0 1 HELLO-1 2082 ABC	UM/MOD, (WORD) (FIND) inside words, MOVE
8	`' : PARENT EXECUTE SEEDED 99 . ; SEEDED`	99	the seed : , run through EXECUTE
9	`VARIABLE THERE2 30000 THERE2 ! : H2 THERE2 @ ; : C2 THERE2 @ ! 2 TH…`	30004 7 5678 0	a space table of Forth words: >CS, vectored , HERE PATCH, (CLOSE)
10	`: CL 0 ['] NOOP 0 >CS (CLOSE) ; CL OPEN? . COMPILING? .`	0 0	>CS and (CLOSE) with space 0 (promote), OPEN? COMPILING?
11	`0 5 3 UM/MOD`	*UM/MOD OVERFLOW*	a trap: UM/MOD OVERFLOW
12	`DEPTH . 77 .`	0 77	after an error: QUIT cleans up ((ABORT))
13	`: LONG 300 BEGIN 65 EMIT 1- DUP 0= UNTIL DROP ; LONG`	AAAAAAAAAAAAAAAAAAAAAAAA…	300 characters: the ring fills, EMIT WAITs
14	`: BAD 1 IF ;`	*; with an unclosed IF, BEGIN or CASE*	; with an IF open: a trap from the compile stack's frame
15	`DROP`	*DATA STACK UNDERFLOW*	a trap: DATA STACK UNDERFLOW
16	`]`	*] without [*	a trap:] without [
17	`1 2 3 . . .`	3 2 1	1 2 3 . . .

`make_ref.js` also counts the calls of every native during the run: **all 41 are called at least once.**

Results

Port	Runs on	Result
x86-64	this computer's CPU, natively (ctypes)	all 17 tests identical: output, errors, all 64K of RAM
x86-64	the unicorn emulator (to count instructions)	all 17 identical
ARM64	unicorn	all 17 identical
RISC-V RV64IM	unicorn	all 17 identical
65C02	py65	all 17 identical, outside the port's own bytes (below)

Is the comparison sharp? Twice, a single instruction was broken on purpose: `+` made to subtract. On x86-64 the very first test failed (157 bytes of RAM differed, and the output was lost to a stack underflow); on the 65C02 all 17 tests failed. The originals pass again. A port that differed anywhere would be caught.

What the 65C02 port keeps for itself, and so is left out of its comparison: its zero-page scratch ($E0–$FC) and the CPU stack (page 1), both inside ITCH's TIB ($0040–$043F), which QUIT never uses; and its code ($C000) with its latches ($BF00), in the unused top of the text area. The 64-bit ports keep their state outside ITCH's 64K, so nothing is left out of theirs.

Known differences from the JavaScript kernel, none reached by the tests:

- The JavaScript kernel stops a line after 2,000,000 steps (MAX STEPS EXCEEDED). That guard is a host feature, not part of the machine; the ports run until they stop, sleep or spend the budget the host gives them.
- When a definition is nested, the JavaScript kernel compares the open frame's space (an unsigned cell) with the new one (a signed number), so a space table at $8000 or above would be refused. The 64-bit ports reproduce that exactly; the 6502 port compares 16 bits and would accept it. Space tables live in the dictionary, below $8000.
- A cell at $FFFF: the JavaScript kernel wraps its high byte to $0000, as the 6502 does by itself. The 64-bit ports allocate a few bytes past 64K instead. No ITCH word reads a cell there.

Running it yourself

```
node make_ref.js                 # the reference, from ITCH-TINY.html
python3 run_proof.py x64        # natively
python3 run_proof.py x64-emu    # x86-64, arm64, rv64 in unicorn
python3 run_proof.py arm64
python3 run_proof.py rv64
python3 run_proof.py 6502       # in py65 ( about 1.5 minutes )
python3 run_proof.py rv64 --count   # and the instruction counts
```

Needs: Node, Python 3, gcc (for the native x86-64 run) and `pip install py65 unicorn ziglang` (ziglang supplies the ARM64 and RISC-V assemblers). Files: `itch-6502.s itch-x64.s itch-arm64.s itch-rv64.s` (the kernels), `asm6502.py` (a two-pass 65C02 assembler whose opcode table is py65's own), `make_ref.js`, `run_proof.py`, `gen_reference.py` (this document).

9. Trade-offs, stated plainly

The primitives are small. The design's costs are in the step around them. On x86-64 a BRANCH step is 30 instructions, and BRANCH's own body is 2 of them. The other 28 are spent like this (counted instruction by instruction in itch-x64.s):

Part of every step	x86-64 instructions	What it buys
the step budget (test, decrement)	3	the host can run the machine in slices and stop it anywhere
NEXT: fetch the xt, step IP, keep it 16 bits	3	a 16-bit machine on a 64-bit CPU
EXECUTE: LEN → CFA → KIND → native number → table entry	11	the xt is the header, so every word has a PARENT and one layout
the calls and returns around EXECUTE and the primitive	4	plain, portable subroutine structure
after the step: RP = 0? STOP? the WAIT latch?	7	errors stop the machine without unwinding; QUIT can return; WAIT

A speed-first port would change these, not the primitives: keep SP and RP in CPU registers (and treat Forth's stores to $0C and $0E specially), keep the top of the stack in a register, make EXIT and HALT leave the loop instead of polling RP and STOP on every step, and give each header a cached code address. For comparison, a classic indirect-threaded NEXT is 2 instructions on x86-64 (`lodsq` / `jmp [rax]`), 3 on ARM64, 4 on RISC-V and about a dozen on a 6502 (fig-Forth's); those are typical figures from other Forths, not measured here.

On the 6502 everything 16-bit takes two of everything, there is no multiply or divide (`*` `/` `UM/MOD` are loops of 595–954 cycles), and every stack move is checked. In return, ITCH's register page is the 6502's zero page, which is the cheapest memory the CPU has, so the port needs no register copying at all.

What the design wins, and why the same 64K image can run unchanged on four very different CPUs:

- no operating system, no allocation, no host features: a kernel is a jump table and 4–5 KB of routines over one byte array;
- the whole state in RAM: stop after any step, resume, inspect, or read it from another machine over a serial cord;
- structure checking and cross-compilation built into four tiny compiler primitives instead of spread through the compiler;
- every error a numbered trap that leaves the stacks as they were.

Native ITC MachineForth

Unlike traditional STC hardware compilers, this engine is a **Native ITC MachineForth**. It combines a kernel of native machine code with the dynamic late-binding and memory efficiency of Indirect Threaded Code.

1 · Native STC MachineForth

What it is. The traditional execution model: built into silicon, as in Chuck Moore's chips, or generated by a Subroutine Threaded Code compiler.

How it works. Words compile directly into native opcodes (`CALL` `JSR` `JMP`). The physical CPU's program counter runs the thread.

The trade-off. Maximum raw speed, but a rigid structure: calls are fixed when they are compiled, and every call is a full instruction in memory.

2 · Native ITC MachineForth (this system)

What it is. A kernel running natively on bare metal that uses Indirect Threaded Code.

How it works. The core, `NEXT`, is native machine code, but it walks a list of execution tokens with a virtual instruction pointer, `IP`.

The advantage. A native kernel, with dynamic late-binding, dense code (one 2-byte token per call, where a native call is 3 bytes on a 6502 and 4–5 on ARM64 or x86-64), clean host-to-target compiling, and threads that run unchanged on any CPU the kernel is written for.

How it ties together

Because `NEXT` walks execution tokens instead of executing hardware `JSR` / `CALL` instructions, the system keeps its late-binding at run time: a deferred word is re-aimed with `IS`, and a word's behaviour can be redirected while the program runs. Two mechanisms carry the host-to-target relationship:

- **The words that act while compiling: the boundary between build time and run time.** ITCH keeps no flag bit: `IMMEDIATE` moves a word into the COMPILER wordlist, and that is what makes it act while a definition is built. Every other word is compiled, as a 2-byte token, into the definition. When the definition is being built in a target machine, the host's compiling words (`IF THEN BEGIN LITERAL`) run on the host and shape the target's memory through the vectored `HERE , PATCH`; the words themselves land in the target.
- **`CREATE … DOES>`: compile-time data bound to run-time behaviour.** `CREATE` runs at build time: it makes a header and lays down the data. `DOES>` stores the address of the code after it in the new word's ACTION cell, so at run time the word pushes its own data address and runs that code, with `SELF` naming the word. In ITCH every such word also records the word that made it: its PARENT.